

Práctica Optimización para Grandes Volúmenes de Datos: Data streaming con Apache Kafka



Miembros del grupo:

Jaime Vaquero Rabahieh	<code>jaime.vaquero@alumnos.upm.es</code>
Pedro Arroyo Urbina	<code>pedro.arroyo@alumnos.upm.es</code>
Zakaria Lasry Sahraoui	<code>z.lsahraoui@alumnos.upm.es</code>
Pablo Chicharro Gómez	<code>p.cgomez@alumnos.upm.es</code>

14 de marzo de 2026

Índice

1. Parte 1: Primeros pasos con Apache Kafka (1 punto)	2
2. Parte 2: Data Streaming usando kafka-python (4 puntos)	2
3. Parte 3: Integrando procesos ML en Data Streaming (2 puntos)	13

Índice de figuras

1. Ejecución Productor Consumidor.	2
2. Ejecución Productor.	3
3. Ejecución Consumidor.	3
4. Gráfico generado por el consumidor.	4
5. Resultado con <code>auto_offset_reset = latest</code>	5
6. Resultado con <code>auto_offset_reset = earliest</code>	6
7. Resultado con <code>auto_offset_reset = none</code>	6
8. Código del productor con un array NumPy.	7
9. Comando para modificar el número de particiones.	8
10. Salida del productor mostrando envíos de múltiples productores.	8
11. Salida de los consumidores.	9
12. Grafica generada del sensor_id 0	10
13. Grafica generada del sensor_id 1	10
14. Grafica generada del sensor_id 2	11
15. Grafica generada del sensor_id 3	11
16. Grafica generada del sensor_id 4	12
17. Grafica generada del sensor_id 5	12
18. Grafica generada del sensor_id 6	13
19. Salida del productor.	14
20. Salida de los consumidores ML.	14
21. Salida del productor.	15
22. Salida del consumidor 1 – ML A (ejercicio 3.3).	16
23. Salida del consumidor 2 – ML B (ejercicio 3.3).	16
24. Salida Nuevo Consumidor.	17
25. Panel de control – Sensor ID 0.	17
26. Panel de control – Sensor ID 1.	18
27. Panel de control – Sensor ID 2.	18
28. Panel de control – Sensor ID 3.	19
29. Panel de control – Sensor ID 4.	19
30. Panel de control – Sensor ID 5.	20
31. Panel de control – Sensor ID 6.	20

Parte 1: Primeros pasos con Apache Kafka (1 punto)

Ejercicio 1.1

Sigue las instrucciones de la presentación para crear un publicador que publique a un tópico de Kafka el contenido escrito por la línea de comando y un suscriptor que reciba esos mensajes. Adjunta capturas de ambas pantallas (1 punto).

Para este primer ejercicio se utilizaron los scripts de consola propios de Kafka:

`kafka-console-producer.sh` para el productor y `kafka-console-consumer.sh` para el consumidor.

El productor se conecta al *broker* en `localhost:9092` y publica al tópico `firstTopic`.

El consumidor se conecta al mismo *broker* y escucha ese tópico en tiempo real. Como puede observarse en la captura, los mensajes escritos en la terminal del productor (“Hola”, “Mundo!”) aparecen inmediatamente en la terminal del consumidor, lo que demuestra el funcionamiento básico del sistema de mensajería.



Figura 1: Ejecución Productor Consumidor.

Parte 2: Data Streaming usando kafka-python (4 puntos)

Ejercicio 2.1

Ejecuta las celdas de los archivos `water_data_producer_pr_2.ipynb` y `water_data_consumer_pr_2.ipynb` y adjunta capturas de los resultados (0.5 puntos)

A continuación mostraremos capturas de pantalla de la ejecución del productor y consumidor. El productor genera mensajes en formato JSON con un *timestamp* y los cuatro valores del sensor, y los envía al tópico `water_quality`. El consumidor se suscribe a ese tópico, deserializa los mensajes y los almacena para su posterior visualización.

■ Productor

```

except KeyboardInterrupt:
    print("Stopped producing messages.")
finally:
    producer.close()

Producing messages to Kafka topic 'water_quality'...
Sent: {'timestamp': 1772358982, 'water_temperature': 27.6421323048369, 'ph_level': 8.184036584695704, 'turbidity': 15.2, 'dissolved_oxygen': 5.4}
Sent: {'timestamp': 1772358983, 'water_temperature': 26.645672727523714, 'ph_level': 8.298411585106635, 'turbidity': 41.06, 'dissolved_oxygen': 6.21}
Sent: {'timestamp': 1772358984, 'water_temperature': 26.73800526223941, 'ph_level': 7.7207888229908095, 'turbidity': 7.55, 'dissolved_oxygen': 10.99}
Sent: {'timestamp': 1772358985, 'water_temperature': 27.51241036740484, 'ph_level': 7.6164994588839, 'turbidity': 44.97, 'dissolved_oxygen': 7.86}
Sent: {'timestamp': 1772358986, 'water_temperature': 28.077240430656644, 'ph_level': 8.295720419888887, 'turbidity': 38.63, 'dissolved_oxygen': 6.19}
Sent: {'timestamp': 1772358987, 'water_temperature': 27.571950380199556, 'ph_level': 8.376229846398298, 'turbidity': 20.14, 'dissolved_oxygen': 9.28}
Sent: {'timestamp': 1772358988, 'water_temperature': 27.611301402172064, 'ph_level': 8.32019726762121, 'turbidity': 10.69, 'dissolved_oxygen': 10.54}
Sent: {'timestamp': 1772358989, 'water_temperature': 27.493187105735817, 'ph_level': 8.025692359596462, 'turbidity': 22.57, 'dissolved_oxygen': 9.63}
Sent: {'timestamp': 1772358990, 'water_temperature': 27.309716491135735, 'ph_level': 8.143187961379649, 'turbidity': 40.22, 'dissolved_oxygen': 11.29}
Sent: {'timestamp': 1772358991, 'water_temperature': 26.537654200937332, 'ph_level': 8.091355838975105, 'turbidity': 21.6, 'dissolved_oxygen': 7.87}
Sent: {'timestamp': 1772358992, 'water_temperature': 27.652912495815364, 'ph_level': 7.770306261951204, 'turbidity': 13.89, 'dissolved_oxygen': 5.53}
Sent: {'timestamp': 1772358993, 'water_temperature': 27.997135483899747, 'ph_level': 8.442092589780408, 'turbidity': 7.56, 'dissolved_oxygen': 11.76}
Sent: {'timestamp': 1772358994, 'water_temperature': 27.519324949043494, 'ph_level': 7.779452924830663, 'turbidity': 27.04, 'dissolved_oxygen': 5.57}
Sent: {'timestamp': 1772358995, 'water_temperature': 26.402366554055735, 'ph_level': 7.969512424551348, 'turbidity': 42.8, 'dissolved_oxygen': 11.76}
Sent: {'timestamp': 1772358996, 'water_temperature': 26.530381780436226, 'ph_level': 8.263385168369966, 'turbidity': 42.07, 'dissolved_oxygen': 11.84}
Sent: {'timestamp': 1772358997, 'water_temperature': 26.62505837689096, 'ph_level': 7.809723814866988, 'turbidity': 36.56, 'dissolved_oxygen': 5.1}
Sent: {'timestamp': 1772358998, 'water_temperature': 27.451796510082936, 'ph_level': 8.447590822300484, 'turbidity': 35.28, 'dissolved_oxygen': 8.18}
Sent: {'timestamp': 1772358999, 'water_temperature': 26.46865473537547, 'ph_level': 8.500236409927437, 'turbidity': 19.03, 'dissolved_oxygen': 10.76}
Sent: {'timestamp': 1772359000, 'water_temperature': 27.615040206167862, 'ph_level': 7.803509489927706, 'turbidity': 25.7, 'dissolved_oxygen': 10.94}
Sent: {'timestamp': 1772359001, 'water_temperature': 28.173510022616206, 'ph_level': 8.499885556378759, 'turbidity': 13.29, 'dissolved_oxygen': 11.11}
Sent: {'timestamp': 1772359002, 'water_temperature': 27.26005998062727, 'ph_level': 8.154708730271011, 'turbidity': 48.75, 'dissolved_oxygen': 6.78}
Sent: {'timestamp': 1772359003, 'water_temperature': 27.826874733844186, 'ph_level': 7.881692816842249, 'turbidity': 26.07, 'dissolved_oxygen': 7.3}
Sent: {'timestamp': 1772359004, 'water_temperature': 26.879583859624308, 'ph_level': 7.714253420127565, 'turbidity': 20.1, 'dissolved_oxygen': 7.72}
Sent: {'timestamp': 1772359005, 'water_temperature': 27.555546512347824, 'ph_level': 7.827554428982738, 'turbidity': 29.67, 'dissolved_oxygen': 8.65}
Sent: {'timestamp': 1772359006, 'water_temperature': 26.36847653244244, 'ph_level': 8.348931466752385, 'turbidity': 48.32, 'dissolved_oxygen': 7.82}
Sent: {'timestamp': 1772359007, 'water_temperature': 26.714460249515405, 'ph_level': 7.541767411898607, 'turbidity': 26.07, 'dissolved_oxygen': 11.19}
Sent: {'timestamp': 1772359008, 'water_temperature': 27.99562076404529, 'ph_level': 7.934107399626357, 'turbidity': 48.78, 'dissolved_oxygen': 6.25}
Sent: {'timestamp': 1772359009, 'water_temperature': 27.758675373340417, 'ph_level': 8.019218960535584, 'turbidity': 33.02, 'dissolved_oxygen': 6.29}
Sent: {'timestamp': 1772359010, 'water_temperature': 28.22147605285034, 'ph_level': 8.15172263408073, 'turbidity': 38.03, 'dissolved_oxygen': 9.09}
Sent: {'timestamp': 1772359011, 'water_temperature': 27.59024342366953, 'ph_level': 8.227281973648491, 'turbidity': 45.82, 'dissolved_oxygen': 9.06}
Sent: {'timestamp': 1772359012, 'water_temperature': 26.372864304574406, 'ph_level': 8.032219183478354, 'turbidity': 20.98, 'dissolved_oxygen': 10.24}

```

Figura 2: Ejecución Productor.

■ Consumidor

```

try:
    while True:
        update_plot(consumer)
except KeyboardInterrupt:
    print("Stopped visualization.")
    consumer.close()

Subscribed to Kafka topic 'water_quality'.
Received: {'timestamp': 1772358992, 'water_temperature': 27.652912495815364, 'ph_level': 7.770306261951204, 'turbidity': 13.89, 'dissolved_oxygen': 5.53}
Received: {'timestamp': 1772358993, 'water_temperature': 27.997135483899747, 'ph_level': 8.442092589780408, 'turbidity': 7.56, 'dissolved_oxygen': 11.76}
Received: {'timestamp': 1772358994, 'water_temperature': 27.519324949043494, 'ph_level': 7.779452924830663, 'turbidity': 27.04, 'dissolved_oxygen': 5.57}
Received: {'timestamp': 1772358995, 'water_temperature': 26.402366554055735, 'ph_level': 7.969512424551348, 'turbidity': 42.8, 'dissolved_oxygen': 11.76}
Received: {'timestamp': 1772358996, 'water_temperature': 26.530381780436226, 'ph_level': 8.263385168369966, 'turbidity': 42.07, 'dissolved_oxygen': 11.84}
Received: {'timestamp': 1772358997, 'water_temperature': 26.62505837689096, 'ph_level': 7.809723814866988, 'turbidity': 36.56, 'dissolved_oxygen': 5.1}
Received: {'timestamp': 1772358998, 'water_temperature': 27.451796510082936, 'ph_level': 8.447590822300484, 'turbidity': 35.28, 'dissolved_oxygen': 8.18}
Received: {'timestamp': 1772358999, 'water_temperature': 26.46865473537547, 'ph_level': 8.500236409927437, 'turbidity': 19.03, 'dissolved_oxygen': 10.76}
Received: {'timestamp': 1772359000, 'water_temperature': 27.615040206167862, 'ph_level': 7.803509489927706, 'turbidity': 25.7, 'dissolved_oxygen': 10.94}
Received: {'timestamp': 1772359001, 'water_temperature': 28.173510022616206, 'ph_level': 8.499885556378759, 'turbidity': 13.29, 'dissolved_oxygen': 11.11}
Received: {'timestamp': 1772359002, 'water_temperature': 27.26005998062727, 'ph_level': 8.154708730271011, 'turbidity': 48.75, 'dissolved_oxygen': 6.78}
Received: {'timestamp': 1772359003, 'water_temperature': 27.826874733844186, 'ph_level': 7.881692816842249, 'turbidity': 26.07, 'dissolved_oxygen': 7.3}
Received: {'timestamp': 1772359004, 'water_temperature': 26.879583859624308, 'ph_level': 7.714253420127565, 'turbidity': 20.1, 'dissolved_oxygen': 7.72}
Received: {'timestamp': 1772359005, 'water_temperature': 27.555546512347824, 'ph_level': 7.827554428982738, 'turbidity': 29.67, 'dissolved_oxygen': 8.65}
Received: {'timestamp': 1772359006, 'water_temperature': 26.36847653244244, 'ph_level': 8.348931466752385, 'turbidity': 48.32, 'dissolved_oxygen': 7.82}
Received: {'timestamp': 1772359007, 'water_temperature': 26.714460249515405, 'ph_level': 7.541767411898607, 'turbidity': 26.07, 'dissolved_oxygen': 11.19}
Received: {'timestamp': 1772359008, 'water_temperature': 27.99562076404529, 'ph_level': 7.934107399626357, 'turbidity': 48.78, 'dissolved_oxygen': 6.25}
Received: {'timestamp': 1772359009, 'water_temperature': 27.758675373340417, 'ph_level': 8.019218960535584, 'turbidity': 33.02, 'dissolved_oxygen': 6.29}
Received: {'timestamp': 1772359010, 'water_temperature': 28.22147605285034, 'ph_level': 8.15172263408073, 'turbidity': 38.03, 'dissolved_oxygen': 9.09}
Received: {'timestamp': 1772359011, 'water_temperature': 27.59024342366953, 'ph_level': 8.227281973648491, 'turbidity': 45.82, 'dissolved_oxygen': 9.06}
Received: {'timestamp': 1772359012, 'water_temperature': 26.372864304574406, 'ph_level': 8.032219183478354, 'turbidity': 20.98, 'dissolved_oxygen': 10.24}

```

Figura 3: Ejecución Consumidor.

Como puede verse, cada mensaje recibido por el consumidor contiene los mismos cuatro campos que el productor envió, confirmando que la serialización y deserialización JSON funciona correctamente en ambos extremos del canal.

Ejercicio 2.2

Adjunta foto del gráfico generado por el consumidor (0.25 puntos).

El consumidor actualiza en tiempo real un gráfico de cuatro subplots (uno por variable). Cada nuevo mensaje recibido añade un punto a la serie temporal correspondiente y redibuja la figura.

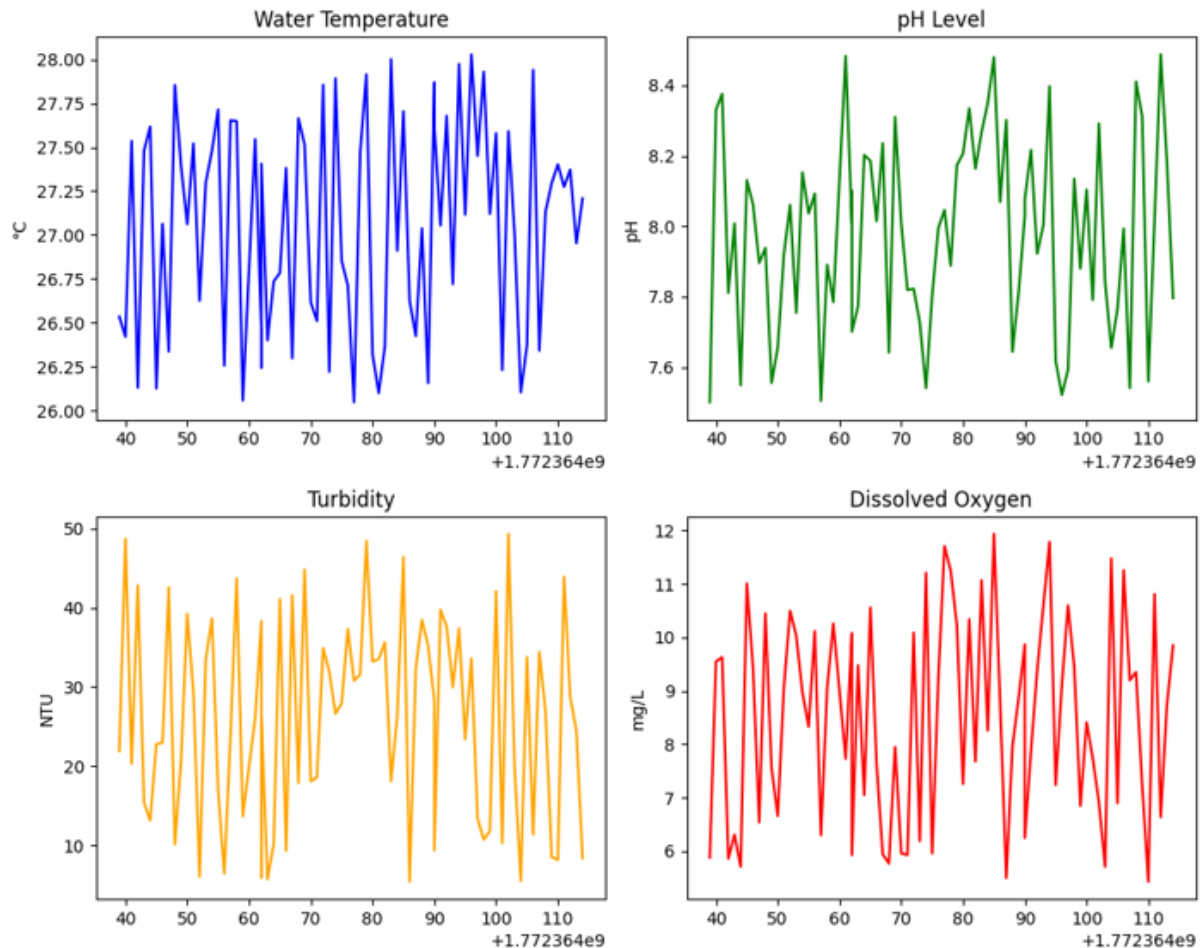


Figura 4: Gráfico generado por el consumidor.

Ejercicio 2.3

En la configuración del consumidor, ¿qué efecto tiene cambiar la propiedad `auto_offset_reset`? Prueba diferentes valores y adjunta tus conclusiones (0.25 puntos).

La propiedad `auto_offset_reset` determina qué offset de lectura utiliza el consumidor cuando no existe un offset confirmado previo para su `group_id` en el tópico. Puede tomar tres valores:

- **latest** (valor por defecto): el consumidor empieza a leer únicamente los mensajes que lleguen *a partir del momento* en que se inicia, ignorando todos los mensajes ya almacenados en el tópico. Esto es adecuado para casos de uso en tiempo real donde solo interesan los datos más recientes.

- **earliest**: el consumidor rebobina hasta el principio del tópic y lee *todos* los mensajes disponibles desde el offset más antiguo. Útil cuando se quiere reprocesar el historial completo.
- **none**: si no existe un offset confirmado para el grupo, Kafka lanza una excepción (`NoOffsetForPartitionError`) en lugar de asumir un comportamiento por defecto. Esto obliga al desarrollador a gestionar explícitamente la posición de lectura.

Por defecto, en **latest** genera un sistema muy parecido al impreso en el ejercicio anterior, donde las cuatro variables se van modificando a lo largo del tiempo:

■ Latest

```
[*]: consumer = initialize_consumer_latest()
print("Subscribed to Kafka topic 'water_quality'.")

try:
    while True:
        update_plot(consumer)
except KeyboardInterrupt:
    print("Stopped visualization.")
    consumer.close()
```

Subscribed to Kafka topic 'water_quality'.

Received: {'timestamp': 1772359317, 'water_temperature': 26.958128335087338, 'ph_level': 7.590377779847329, 'turbidity': 23.04, 'dissolved_oxygen': 6.18}

Received: {'timestamp': 1772359318, 'water_temperature': 27.55038069071203, 'ph_level': 8.52649443121881, 'turbidity': 17.06, 'dissolved_oxygen': 5.3}

Received: {'timestamp': 1772359319, 'water_temperature': 28.111552236744195, 'ph_level': 8.247309425670988, 'turbidity': 17.91, 'dissolved_oxygen': 8.16}

Received: {'timestamp': 1772359320, 'water_temperature': 27.416512135450894, 'ph_level': 7.645660259197564, 'turbidity': 34.12, 'dissolved_oxygen': 6.34}

Received: {'timestamp': 1772359321, 'water_temperature': 27.96628867600948, 'ph_level': 7.9281948793961945, 'turbidity': 25.3, 'dissolved_oxygen': 8.6}

Received: {'timestamp': 1772359322, 'water_temperature': 26.405231636648786, 'ph_level': 8.490753963255198, 'turbidity': 21.47, 'dissolved_oxygen': 10.67}

Received: {'timestamp': 1772359323, 'water_temperature': 27.475877504448775, 'ph_level': 8.129457515862986, 'turbidity': 39.67, 'dissolved_oxygen': 10.01}

Received: {'timestamp': 1772359324, 'water_temperature': 26.81336565057821, 'ph_level': 8.47834227441013, 'turbidity': 8.19, 'dissolved_oxygen': 7.66}

Received: {'timestamp': 1772359325, 'water_temperature': 27.15394510680154, 'ph_level': 8.121998695842706, 'turbidity': 39.33, 'dissolved_oxygen': 7.53}

Received: {'timestamp': 1772359326, 'water_temperature': 26.872427676142603, 'ph_level': 8.394153449998795, 'turbidity': 31.27, 'dissolved_oxygen': 11.31}

Received: {'timestamp': 1772359327, 'water_temperature': 27.32238844193971, 'ph_level': 8.090732539233892, 'turbidity': 14.75, 'dissolved_oxygen': 9.37}

Received: {'timestamp': 1772359328, 'water_temperature': 27.77929589252942, 'ph_level': 7.559893198256376, 'turbidity': 45.7, 'dissolved_oxygen': 11.83}

Received: {'timestamp': 1772359329, 'water_temperature': 26.77862908944495, 'ph_level': 8.373328785652504, 'turbidity': 19.6, 'dissolved_oxygen': 6.75}

Received: {'timestamp': 1772359330, 'water_temperature': 26.372364093616593, 'ph_level': 8.501904241677682, 'turbidity': 35.6, 'dissolved_oxygen': 9.57}

Received: {'timestamp': 1772359331, 'water_temperature': 27.909808591891274, 'ph_level': 7.531261713056138, 'turbidity': 6.72, 'dissolved_oxygen': 9.0}

Figura 5: Resultado con `auto_offset_reset = latest`.

■ Earliest

En cambio, al modificar esta constante con **earliest** la estructura de las gráficas cambia drásticamente:

Esto se debe a que con **latest** el consumidor recibe siempre los mensajes más recientes primero y con **earliest** los más antiguos. Por lo tanto, si hay algún mensaje recibido más tarde que los demás, con **latest** se leerá cuando es debido, mientras que con **earliest** no. Por eso, las gráficas de **latest** mantienen su orden aún con fallos y las de **earliest** no, siendo la primera la mejor opción para este caso de uso donde los datos se generan de forma continua y ordenada.

```
[*]: consumer = initialize_consumer_earliest()
print("Subscribed to Kafka topic 'water_quality'.")

try:
    while True:
        update_plot(consumer)
except KeyboardInterrupt:
    print("Stopped visualization.")
    consumer.close()
```

Subscribed to Kafka topic 'water_quality'.

Received: {'timestamp': 1772358982, 'water_temperature': 27.6421323048369, 'ph_level': 8.184036584695704, 'turbidity': 15.2, 'dissolved_oxygen': 5.4}

Received: {'timestamp': 1772358983, 'water_temperature': 26.645672727523714, 'ph_level': 8.298411585106635, 'turbidity': 41.06, 'dissolved_oxygen': 6.21}

Received: {'timestamp': 1772358984, 'water_temperature': 26.73800526223941, 'ph_level': 7.7207888229908095, 'turbidity': 7.55, 'dissolved_oxygen': 10.99}

Received: {'timestamp': 1772358985, 'water_temperature': 27.51241036740484, 'ph_level': 7.6164994588839, 'turbidity': 44.97, 'dissolved_oxygen': 7.86}

Received: {'timestamp': 1772358986, 'water_temperature': 28.077240430656644, 'ph_level': 8.295720419888887, 'turbidity': 38.63, 'dissolved_oxygen': 6.19}

Received: {'timestamp': 1772358987, 'water_temperature': 27.571950380199556, 'ph_level': 8.376229846398298, 'turbidity': 20.14, 'dissolved_oxygen': 9.28}

Received: {'timestamp': 1772358988, 'water_temperature': 27.611301402172064, 'ph_level': 8.32019726762121, 'turbidity': 10.69, 'dissolved_oxygen': 10.54}

Received: {'timestamp': 1772358989, 'water_temperature': 27.493187105735817, 'ph_level': 8.025692359596462, 'turbidity': 22.57, 'dissolved_oxygen': 9.63}

Received: {'timestamp': 1772358990, 'water_temperature': 27.309716491135735, 'ph_level': 8.143187961379649, 'turbidity': 40.22, 'dissolved_oxygen': 11.29}

Received: {'timestamp': 1772358991, 'water_temperature': 26.537654200937332, 'ph_level': 8.091355838975105, 'turbidity': 21.6, 'dissolved_oxygen': 7.87}

Received: {'timestamp': 1772358992, 'water_temperature': 27.652912495815364, 'ph_level': 7.770306261951204, 'turbidity': 13.89, 'dissolved_oxygen': 5.53}

Received: {'timestamp': 1772358993, 'water_temperature': 27.997135483899747, 'ph_level': 8.442092589780408, 'turbidity': 7.56, 'dissolved_oxygen': 11.76}

Received: {'timestamp': 1772358994, 'water_temperature': 27.519324949043494, 'ph_level': 7.779452924830663, 'turbidity': 27.04, 'dissolved_oxygen': 5.57}

Received: {'timestamp': 1772358995, 'water_temperature': 26.402366554055735, 'ph_level': 7.969512424551348, 'turbidity': 42.8, 'dissolved_oxygen': 11.7}

Figura 6: Resultado con `auto_offset_reset = earliest`.

■ None

Con el valor `none`, al no existir un offset confirmado previo para el grupo, Kafka lanza inmediatamente una excepción `NoOffsetForPartitionError`, interrumpiendo la ejecución del consumidor. Este comportamiento fuerza al programador a establecer explícitamente el offset antes de iniciar la lectura, siendo una opción adecuada solo en entornos donde el control manual del offset es imprescindible.

```
File /usr/local/lib/python3.10/dist-packages/kafka/consumer/group.py:680, in KafkaConsumer._poll_once(self, timeout_ms, max_records, update_offsets)
677 # Fetch positions if we have partitions we're subscribed to that we
678 # don't know the offset for
679 if not self._subscription.has_all_fetch_positions():
--> 680     self._update_fetch_positions(self._subscription.missing_fetch_positions())
682 # If data is available already, e.g. from a previous network client
683 # poll() call to commit, then just return it immediately
684 records, partial = self._fetcher.fetched_records(max_records, update_offsets=update_offsets)

File /usr/local/lib/python3.10/dist-packages/kafka/consumer/group.py:1112, in KafkaConsumer._update_fetch_positions(self, partitions)
1109 self._coordinator.refresh_committed_offsets_if_needed()
1111 # Then, do any offset lookups in case some positions are not known
-> 1112 self._fetcher.update_fetch_positions(partitions)

File /usr/local/lib/python3.10/dist-packages/kafka/consumer/fetcher.py:186, in Fetcher.update_fetch_positions(self, partitions)
182 elif self._subscriptions.assignment[tp].committed is None:
183     # there's no committed position, so we need to reset with the
184     # default strategy
185     self._subscriptions.need_offset_reset(tp)
--> 186     self._reset_offset(tp)
187 else:
188     committed = self._subscriptions.assignment[tp].committed.offset

File /usr/local/lib/python3.10/dist-packages/kafka/consumer/fetcher.py:233, in Fetcher._reset_offset(self, partition)
231 strategy = 'latest'
232 else:
--> 233     raise NoOffsetForPartitionError(partition)
235 log.debug("Resetting offset for partition %s to %s offset.",
236           partition, strategy)
237 offsets = self._retrieve_offsets({partition: timestamp})

NoOffsetForPartitionError: NoOffsetForPartitionError: TopicPartition(topic='water_quality', partition=0)
```

Figura 7: Resultado con `auto_offset_reset = none`.

Ejercicio 2.4

¿Qué ocurre si intentamos mandar a través del productor un array de NumPy (`numpy.array`)? Adjunta una imagen probándolo y reflexiona sobre los resultados (0.25 puntos).



```

228     check_circular and allow_nan and
229     cls is None and indent is None and separators is None and
230     default is None and not sort_keys and not kw):
--> 231     return _default_encoder.encode(obj)
232 if cls is None:
233     cls = JSONEncoder

File /usr/lib/python3.10/json/encoder.py:199, in JSONEncoder.encode(self, o)
195     return encode_basestring(o)
196 # This doesn't pass the iterator directly to ''.join() because the
197 # exceptions aren't as detailed. The list call should be roughly
198 # equivalent to the PySequence_Fast that ''.join() would do.
--> 199 chunks = self.iterencode(o, _one_shot=True)
200 if not isinstance(chunks, (list, tuple)):
201     chunks = list(chunks)

File /usr/lib/python3.10/json/encoder.py:257, in JSONEncoder.iterencode(self, o, _one_shot)
252 else:
253     _iterencode = _make_iterencode(
254         markers, self.default, _encoder, self.indent, floatstr,
255         self.key_separator, self.item_separator, self.sort_keys,
256         self.skipkeys, _one_shot)
--> 257 return _iterencode(o, 0)

File /usr/lib/python3.10/json/encoder.py:179, in JSONEncoder.default(self, o)
160 def default(self, o):
161     """Implement this method in a subclass such that it returns
162     a serializable object for ``o``, or calls the base implementation
163     (to raise a ``TypeError``).
164     (...)
167     """
168     """
--> 179     raise TypeError(f'Object of type {o.__class__.__name__} '
180                     f'is not JSON serializable')

TypeError: Object of type ndarray is not JSON serializable

```

Figura 8: Código del productor con un array NumPy.

Al intentar enviar un `numpy.array` como parte del mensaje, el serializador JSON de Python lanza un `TypeError: Object of type ndarray is not JSON serializable`. Esto se debe a que el codificador JSON estándar de Python únicamente soporta tipos nativos del lenguaje (listas, diccionarios, enteros, flotantes, cadenas, booleanos y `None`), y los objetos de NumPy no forman parte de ese conjunto.

Para resolver este problema existen varias alternativas: (1) convertir el array a una lista de Python con `.tolist()` antes de incluirlo en el mensaje; (2) implementar un codificador JSON personalizado que extienda `json.JSONEncoder` y gestione los tipos de NumPy; o (3) utilizar un formato de serialización binaria como `pickle` o `msgpack`, aunque esto requiere el deserializador correspondiente en el consumidor. En general, la solución más sencilla y compatible es la conversión a lista nativa de Python.

Ejercicio 2.5

Modifica el tópico para tener 2 particiones. Adjunta captura de la descripción resultante del tópico (0.25 puntos).

Las particiones son la unidad fundamental de paralelismo en Kafka. Cada tópico se divide en una o más particiones, y cada partición es una secuencia ordenada e inmutable de mensajes. Al aumentar el número de particiones de 1 a 2, habilitamos que dos consumidores dentro del mismo grupo puedan leer del tópico simultáneamente (uno por partición), incrementando el throughput del sistema. Además, las particiones permiten distribuir la carga entre distintos brokers en un clúster multi-nodo.


```
# ./kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --alter --topic water_quality --partitions 2
# ./kafka/bin/kafka-topics.sh --list --bootstrap-server localhost:9092
firstTopic
water_quality
# ./kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --describe --topic water_quality
Topic: water_quality TopicId: skJ3myUhs0Kkh0lj7CqBMg PartitionCount: 2 ReplicationFactor: 1 Configs:
    Topic: water_quality Partition: 0 Leader: 0 Replicas: 0 Isr: 0
    Topic: water_quality Partition: 1 Leader: 0 Replicas: 0 Isr: 0
#
```

Figura 9: Comando para modificar el número de particiones.

Como se observa en la descripción del tópic, `PartitionCount: 2` confirma la modificación. Cada partición tiene un `Leader` asignado y `Replicas: 0` dado que solo existe un único broker en este entorno de desarrollo local.

Ejercicio 2.6

Modifica el archivo `water_data_producer_pr_2.ipynb` para que se creen múltiples productores con un id de sensor único que se envíe como *key* del mensaje. Adjunta captura en la que se muestre el envío de mensajes de varios productores en el mismo notebook (1.25 puntos).

Se han creado 7 productores (sensor IDs del 0 al 6) que se ejecutan en paralelo. Cada productor envía su `sensor_id` como *key* del mensaje Kafka, lo que garantiza que todos los mensajes del mismo sensor sean enrutados siempre a la misma partición. Esto es fundamental para preservar el orden temporal de los datos de cada sensor: Kafka solo garantiza el orden dentro de una misma partición, por lo que el uso de la clave del sensor como *key* es la estrategia correcta para este tipo de flujo de datos.

Se imprime el id y el `sensor_data` de 7 en 7:

```
for i in range(producer_number):
    producers[i].close()

[producer=0] key=0 value={'sensor_id': '0', 'timestamp': 1772359998, 'water_temperature': 27.573206739013383, 'ph_level': 8.0608549617704, 'turbidity': 7.15, 'dissolved_oxygen': 9.25}
[producer=1] key=1 value={'sensor_id': '1', 'timestamp': 1772359998, 'water_temperature': 27.896971835095133, 'ph_level': 8.420765955154012, 'turbidity': 38.62, 'dissolved_oxygen': 7.17}
[producer=2] key=2 value={'sensor_id': '2', 'timestamp': 1772359998, 'water_temperature': 27.012253551516128, 'ph_level': 8.511807429546465, 'turbidity': 41.62, 'dissolved_oxygen': 8.52}
[producer=3] key=3 value={'sensor_id': '3', 'timestamp': 1772359998, 'water_temperature': 26.304727470128228, 'ph_level': 8.007073066859938, 'turbidity': 16.33, 'dissolved_oxygen': 5.19}
[producer=4] key=4 value={'sensor_id': '4', 'timestamp': 1772359998, 'water_temperature': 27.64885774498348, 'ph_level': 8.008379270896963, 'turbidity': 27.25, 'dissolved_oxygen': 9.43}
[producer=5] key=5 value={'sensor_id': '5', 'timestamp': 1772359998, 'water_temperature': 27.162961167291986, 'ph_level': 8.121899502670889, 'turbidity': 30.73, 'dissolved_oxygen': 10.73}
[producer=6] key=6 value={'sensor_id': '6', 'timestamp': 1772359998, 'water_temperature': 26.30490127166977, 'ph_level': 8.12726703737788, 'turbidity': 17.21, 'dissolved_oxygen': 6.28}
[producer=0] key=0 value={'sensor_id': '0', 'timestamp': 1772359999, 'water_temperature': 27.89524013465743, 'ph_level': 8.039312887165494, 'turbidity': 48.13, 'dissolved_oxygen': 5.05}
[producer=1] key=1 value={'sensor_id': '1', 'timestamp': 1772359999, 'water_temperature': 27.425468201892002, 'ph_level': 8.152454509095518, 'turbidity': 26.8, 'dissolved_oxygen': 10.63}
[producer=2] key=2 value={'sensor_id': '2', 'timestamp': 1772359999, 'water_temperature': 27.76700562028662, 'ph_level': 8.50676271382376, 'turbidity': 13.69, 'dissolved_oxygen': 7.96}
[producer=3] key=3 value={'sensor_id': '3', 'timestamp': 1772359999, 'water_temperature': 27.365697668342563, 'ph_level': 8.040856071582366, 'turbidity': 45.44, 'dissolved_oxygen': 9.45}
[producer=4] key=4 value={'sensor_id': '4', 'timestamp': 1772359999, 'water_temperature': 28.08659443754896, 'ph_level': 7.9920746681942125, 'turbidity': 18.44, 'dissolved_oxygen': 11.53}
[producer=5] key=5 value={'sensor_id': '5', 'timestamp': 1772359999, 'water_temperature': 27.233216802500642, 'ph_level': 7.756632460750725, 'turbidity': 26.06, 'dissolved_oxygen': 7.76}
[producer=6] key=6 value={'sensor_id': '6', 'timestamp': 1772359999, 'water_temperature': 27.92927193861895, 'ph_level': 8.202004859226758, 'turbidity': 44.76, 'dissolved_oxygen': 10.11}
[producer=0] key=0 value={'sensor_id': '0', 'timestamp': 1772360000, 'water_temperature': 26.333404234940435, 'ph_level': 7.618276339056579, 'turbidity': 38.83, 'dissolved_oxygen': 11.79}
[producer=1] key=1 value={'sensor_id': '1', 'timestamp': 1772360000, 'water_temperature': 26.944341316235217, 'ph_level': 7.624192743481773, 'turbidity': 30.07, 'dissolved_oxygen': 11.22}
[producer=2] key=2 value={'sensor_id': '2', 'timestamp': 1772360000, 'water_temperature': 26.852443205879432, 'ph_level': 7.686026022734743, 'turbidity': 31.16, 'dissolved_oxygen': 10.3}
```

Figura 10: Salida del productor mostrando envíos de múltiples productores.

Ejercicio 2.7

Modifica el archivo `water_data_consumer_pr_2.ipynb` para tener consumidores con un `group_id` común. Cada `sensor_id` debe producir un plot con los datos producidos por ese sensor. Crea una copia del consumidor modificado y adjunta captura de los dos notebooks ejecutándose a la vez gestionando diferentes `sensor_id` y los plots resultantes (1.25 puntos).

Al asignar el mismo `group_id` a los dos consumidores, Kafka los agrupa en un Consumer Group. Cuando un grupo consume un tópico con múltiples particiones, Kafka distribuye automáticamente las particiones entre los consumidores del grupo mediante el proceso de *rebalancing*: cada partición es asignada a exactamente un consumidor del grupo. Con 2 particiones y 2 consumidores, cada consumidor lee de una partición distinta, lo que explica que cada notebook reciba un subconjunto diferente de sensor IDs (los sensores cuya clave fue enrutada a su partición asignada).

```

except KeyboardInterrupt:
    print("Stopped visualization.")
    consumer.close()

Subscribed to Kafka topic 'water_quality'.
Received: {'sensor_id': '6', 'timestamp': 1772360896, 'water_temperature': 27.385648096420276, 'p_h_level': 8.27972277854317, 'turbidity': 42.82, 'dissolved_oxygen': 10.43}
Received: {'sensor_id': '5', 'timestamp': 1772360896, 'water_temperature': 28.13798836027214, 'p_h_level': 8.263678450594993, 'turbidity': 23.48, 'dissolved_oxygen': 6.48}
Received: {'sensor_id': '0', 'timestamp': 1772360897, 'water_temperature': 27.8328604247535, 'p_h_level': 7.813523677372165, 'turbidity': 41.35, 'dissolved_oxygen': 6.59}
Received: {'sensor_id': '2', 'timestamp': 1772360897, 'water_temperature': 26.96852545700419, 'p_h_level': 8.3766870887883, 'turbidity': 6.56, 'dissolved_oxygen': 6.66}
Received: {'sensor_id': '5', 'timestamp': 1772360897, 'water_temperature': 26.338545211675495, 'p_h_level': 8.176852375384842, 'turbidity': 44.78, 'dissolved_oxygen': 8.97}
Received: {'sensor_id': '6', 'timestamp': 1772360897, 'water_temperature': 27.407259965579147, 'p_h_level': 7.63359485136391, 'turbidity': 42.0, 'dissolved_oxygen': 5.09}
Received: {'sensor_id': '0', 'timestamp': 1772360898, 'water_temperature': 27.260805198585512, 'p_h_level': 7.969560952194685, 'turbidity': 40.87, 'dissolved_oxygen': 11.05}
Received: {'sensor_id': '6', 'timestamp': 1772360898, 'water_temperature': 27.086076933950196, 'p_h_level': 7.5438084063905485, 'turbidity': 26.85, 'dissolved_oxygen': 7.61}
Received: {'sensor_id': '2', 'timestamp': 1772360898, 'water_temperature': 27.42456895598778, 'p_h_level': 7.983904923413086, 'turbidity': 13.25, 'dissolved_oxygen': 8.59}
Received: {'sensor_id': '5', 'timestamp': 1772360898, 'water_temperature': 28.091580690886552, 'p_h_level': 8.465583036640373, 'turbidity': 23.85, 'dissolved_oxygen': 10.41}
Received: {'sensor_id': '0', 'timestamp': 1772360899, 'water_temperature': 27.590829258296136, 'p_h_level': 7.988754580268086, 'turbidity': 22.89, 'dissolved_oxygen': 6.74}
Received: {'sensor_id': '5', 'timestamp': 1772360899, 'water_temperature': 27.866226446909415, 'p_h_level': 8.474391988506182, 'turbidity': 28.79, 'dissolved_oxygen': 8.88}
Received: {'sensor_id': '2', 'timestamp': 1772360899, 'water_temperature': 26.48956283493875, 'p_h_level': 7.721080060405664, 'turbidity': 19.56, 'dissolved_oxygen': 10.46}
Received: {'sensor_id': '6', 'timestamp': 1772360899, 'water_temperature': 26.206910535043775, 'p_h_level': 7.546646901164736, 'turbidity': 28.73, 'dissolved_oxygen': 7.22}
Received: {'sensor_id': '0', 'timestamp': 1772360900, 'water_temperature': 28.152652282668516, 'p_h_level': 7.740759057948914, 'turbidity': 8.09, 'dissolved_oxygen': 10.54}
Received: {'sensor_id': '2', 'timestamp': 1772360900, 'water_temperature': 27.723814978928388, 'p_h_level': 7.83689440225362, 'turbidity': 46.7, 'dissolved_oxygen': 5.3}
Received: {'sensor_id': '6', 'timestamp': 1772360900, 'water_temperature': 27.2384412959343, 'p_h_level': 7.829083047452056, 'turbidity': 49.48, 'dissolved_oxygen': 6.12}

Subscribed to Kafka topic 'water_quality'.
Received: {'sensor_id': '1', 'timestamp': 1772360896, 'water_temperature': 26.3439432819191994, 'p_h_level': 7.914312204638359, 'turbidity': 43.02, 'dissolved_oxygen': 5.37}
Received: {'sensor_id': '3', 'timestamp': 1772360896, 'water_temperature': 26.950860177244962, 'p_h_level': 7.793318271060097, 'turbidity': 42.39, 'dissolved_oxygen': 9.96}
Received: {'sensor_id': '4', 'timestamp': 1772360896, 'water_temperature': 27.068706761724194, 'p_h_level': 7.749216762489081, 'turbidity': 40.87, 'dissolved_oxygen': 8.6}
Received: {'sensor_id': '0', 'timestamp': 1772360896, 'water_temperature': 26.642878041918687, 'p_h_level': 7.995702963150163, 'turbidity': 33.64, 'dissolved_oxygen': 8.95}
Received: {'sensor_id': '2', 'timestamp': 1772360896, 'water_temperature': 26.640615267676093, 'p_h_level': 7.768697706074507, 'turbidity': 41.72, 'dissolved_oxygen': 8.93}
Received: {'sensor_id': '1', 'timestamp': 1772360897, 'water_temperature': 27.694893502722294, 'p_h_level': 8.301417384631405, 'turbidity': 9.99, 'dissolved_oxygen': 8.83}
Received: {'sensor_id': '3', 'timestamp': 1772360897, 'water_temperature': 27.91384107725549, 'p_h_level': 8.303723174805338, 'turbidity': 37.7, 'dissolved_oxygen': 7.22}
Received: {'sensor_id': '4', 'timestamp': 1772360897, 'water_temperature': 27.15628927014359, 'p_h_level': 8.43161693119792, 'turbidity': 43.28, 'dissolved_oxygen': 10.70}
Received: {'sensor_id': '3', 'timestamp': 1772360898, 'water_temperature': 26.801871944306647, 'p_h_level': 7.913156867129515, 'turbidity': 25.98, 'dissolved_oxygen': 7.03}
Received: {'sensor_id': '1', 'timestamp': 1772360898, 'water_temperature': 26.710534920804484, 'p_h_level': 8.48939375068347, 'turbidity': 15.2, 'dissolved_oxygen': 5.57}
Received: {'sensor_id': '4', 'timestamp': 1772360898, 'water_temperature': 27.01358612464479, 'p_h_level': 8.176429867009153, 'turbidity': 31.28, 'dissolved_oxygen': 11.88}
Received: {'sensor_id': '1', 'timestamp': 1772360899, 'water_temperature': 27.683689037108234, 'p_h_level': 8.453241271075244, 'turbidity': 30.03, 'dissolved_oxygen': 7.74}
Received: {'sensor_id': '4', 'timestamp': 1772360899, 'water_temperature': 26.90342977264373, 'p_h_level': 7.7951951599535824, 'turbidity': 21.51, 'dissolved_oxygen': 10.28}
Received: {'sensor_id': '3', 'timestamp': 1772360899, 'water_temperature': 28.169501473433982, 'p_h_level': 8.420854450982816, 'turbidity': 25.75, 'dissolved_oxygen': 5.16}
Received: {'sensor_id': '1', 'timestamp': 1772360900, 'water_temperature': 26.31955802030784, 'p_h_level': 8.46409209629079, 'turbidity': 24.54, 'dissolved_oxygen': 9.31}
Received: {'sensor_id': '3', 'timestamp': 1772360900, 'water_temperature': 27.709574077839726, 'p_h_level': 7.796722598265641, 'turbidity': 40.16, 'dissolved_oxygen': 11.88}
Received: {'sensor_id': '4', 'timestamp': 1772360900, 'water_temperature': 26.259996213067367, 'p_h_level': 7.696803870562242, 'turbidity': 29.15, 'dissolved_oxygen': 11.65}
Received: {'sensor_id': '1', 'timestamp': 1772360901, 'water_temperature': 27.087241307609332, 'p_h_level': 8.46409209629079, 'turbidity': 24.54, 'dissolved_oxygen': 9.31}

```

Figura 11: Salida de los consumidores.

Como se puede observar, los sensores obtenidos son distintos según la partición. Esto demuestra el balanceo de carga automático de Kafka: ambos consumidores trabajan en paralelo procesando diferentes subconjuntos de los datos, lo que duplica el throughput del sistema respecto a usar un solo consumidor. A continuación se presentan los 7 plots generados, uno por sensor:

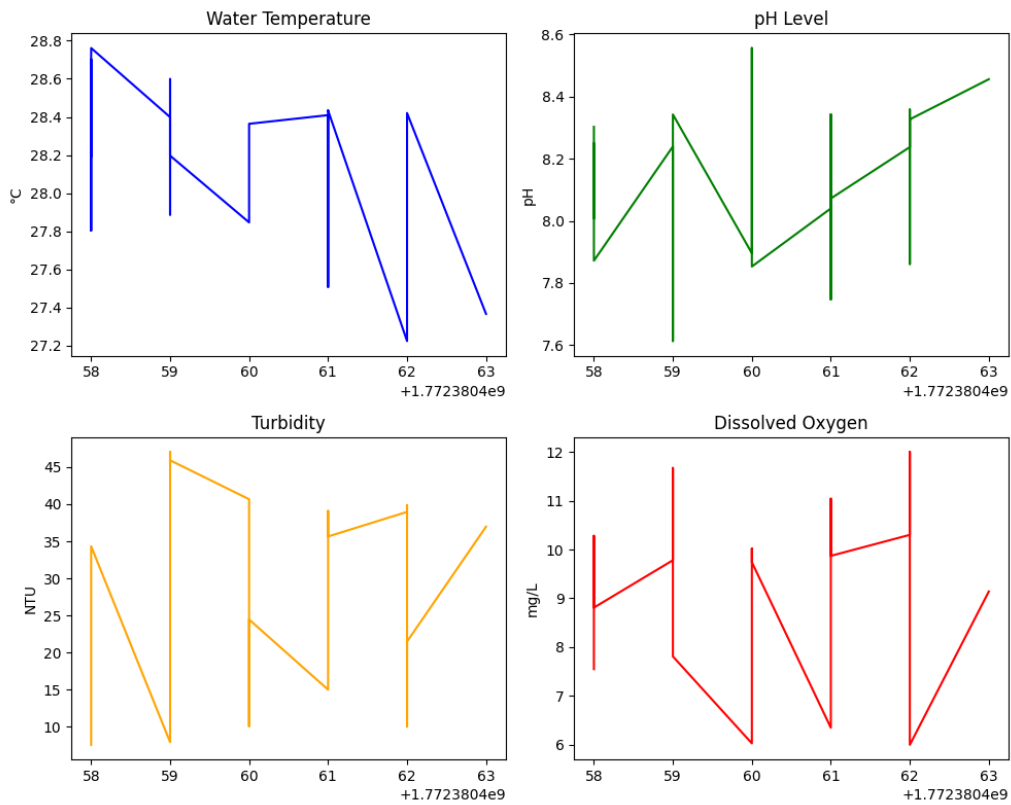


Figura 12: Grafica generada del sensor_id 0

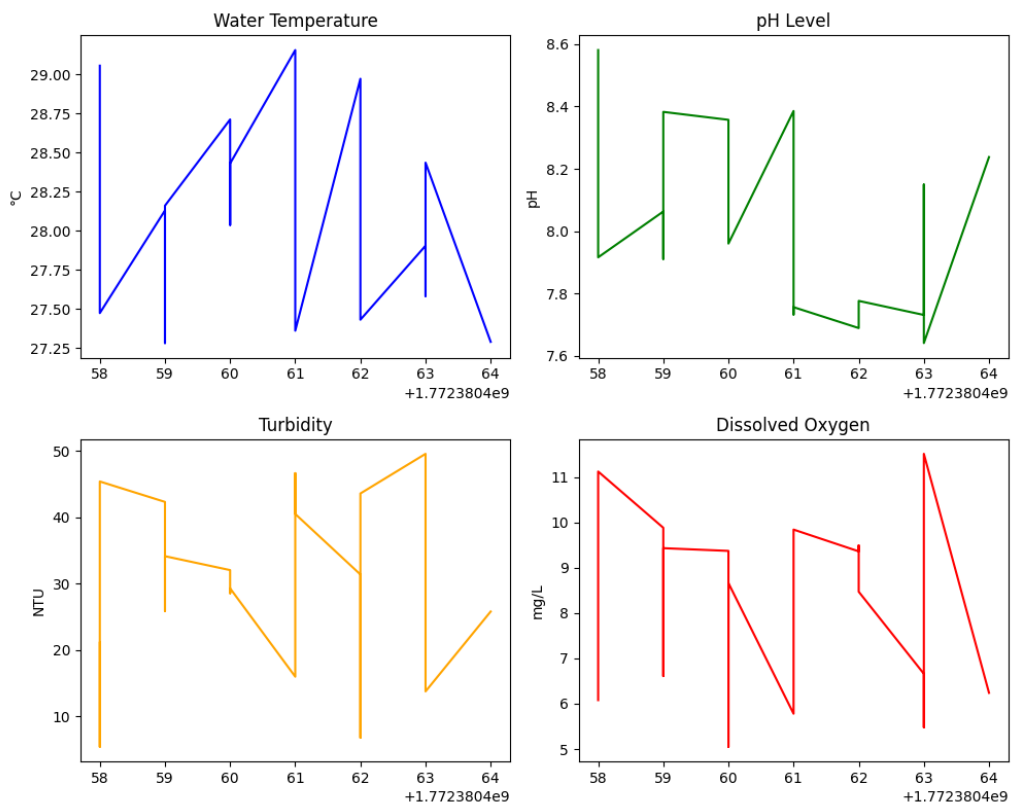


Figura 13: Grafica generada del sensor_id 1

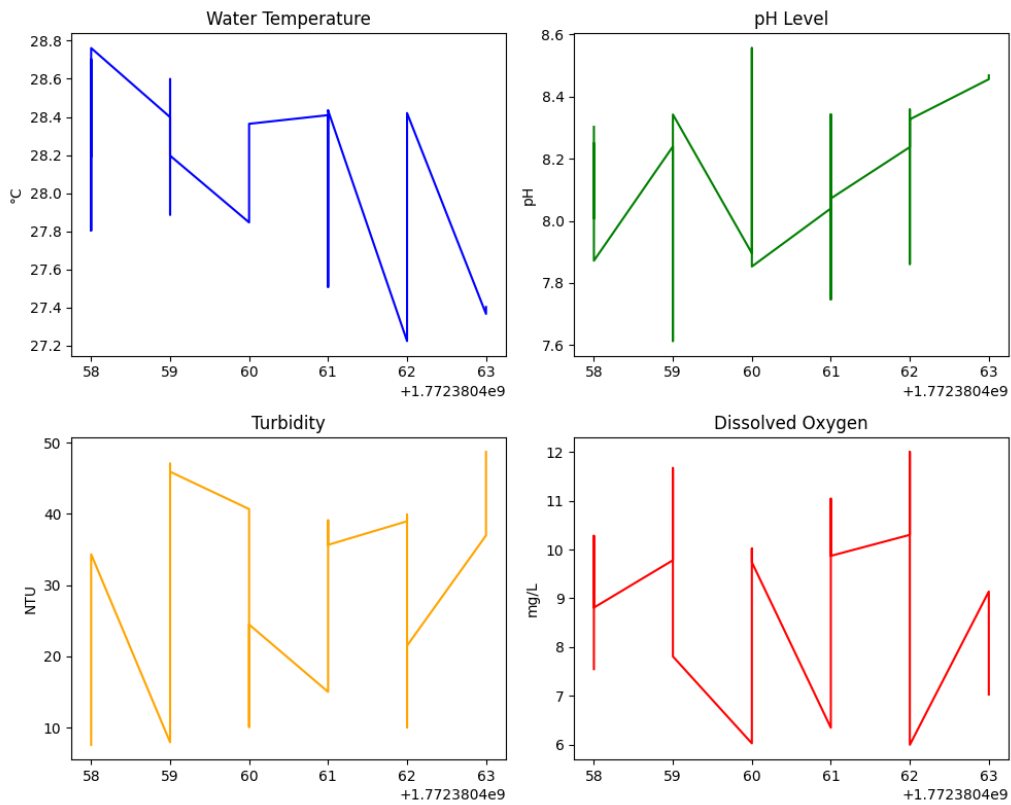


Figura 14: Grafica generada del sensor_id 2

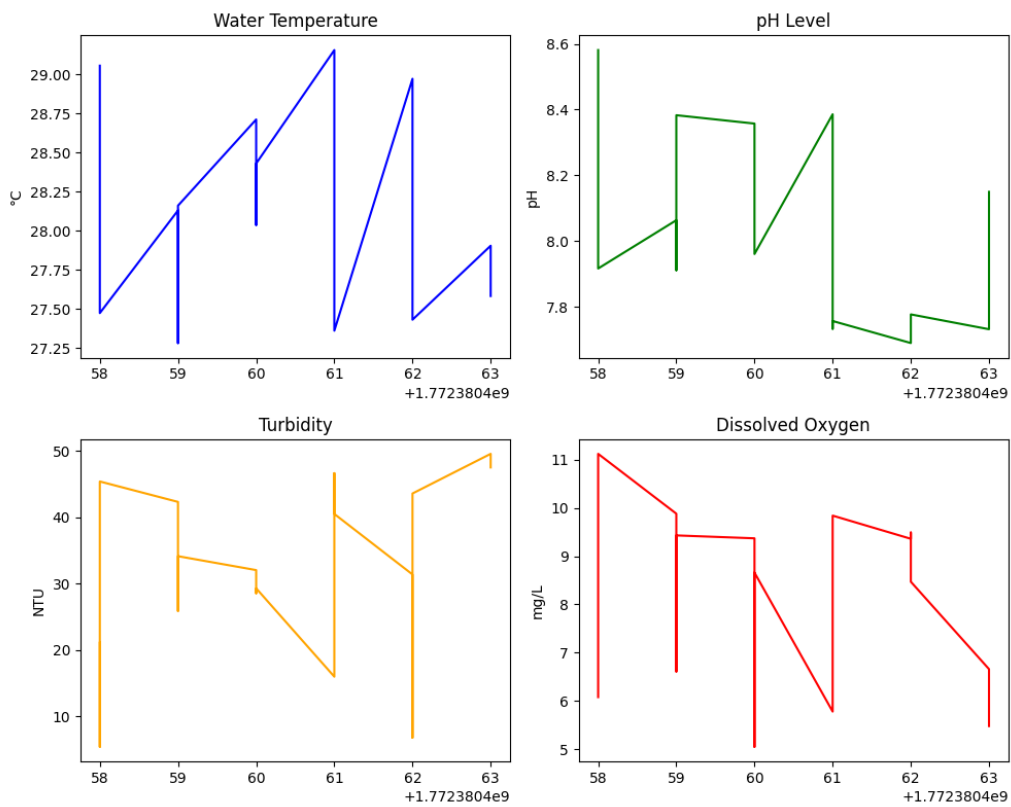


Figura 15: Grafica generada del sensor_id 3

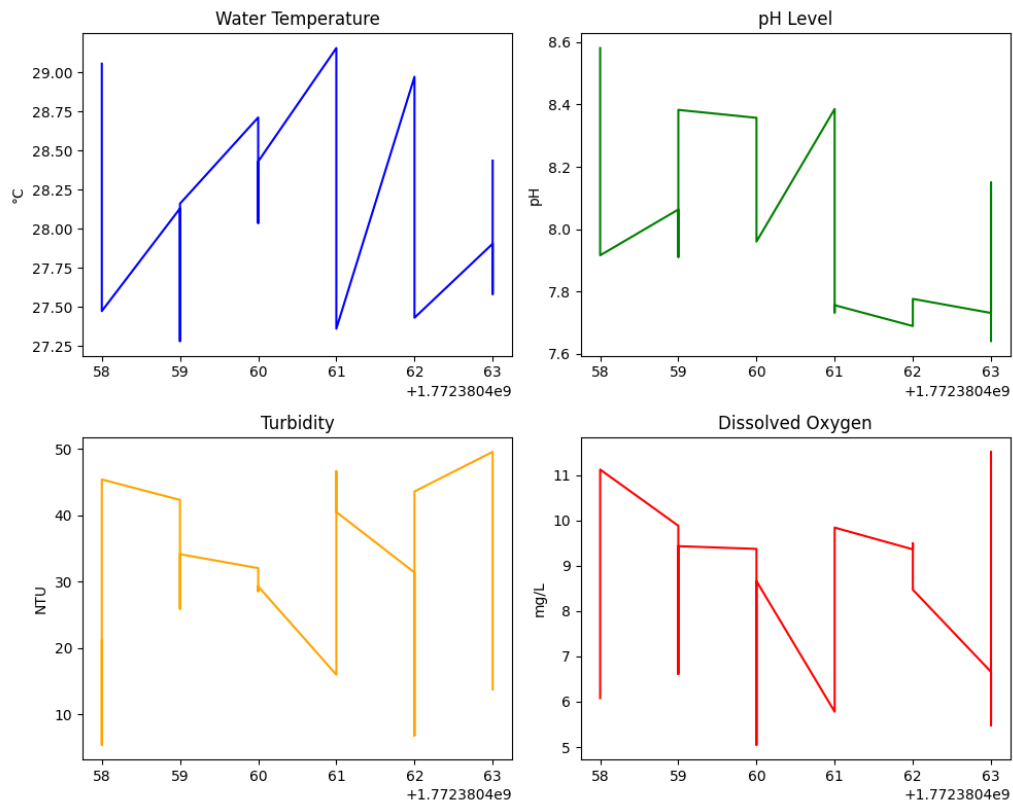


Figura 16: Grafica generada del sensor_id 4

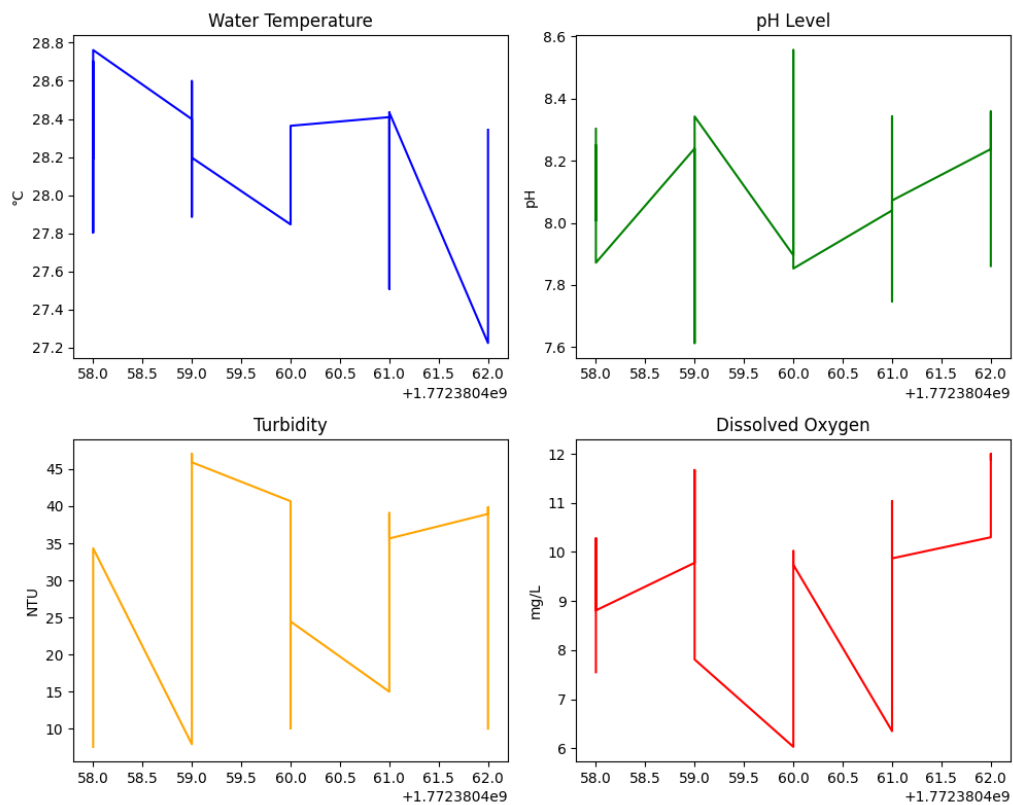


Figura 17: Grafica generada del sensor_id 5

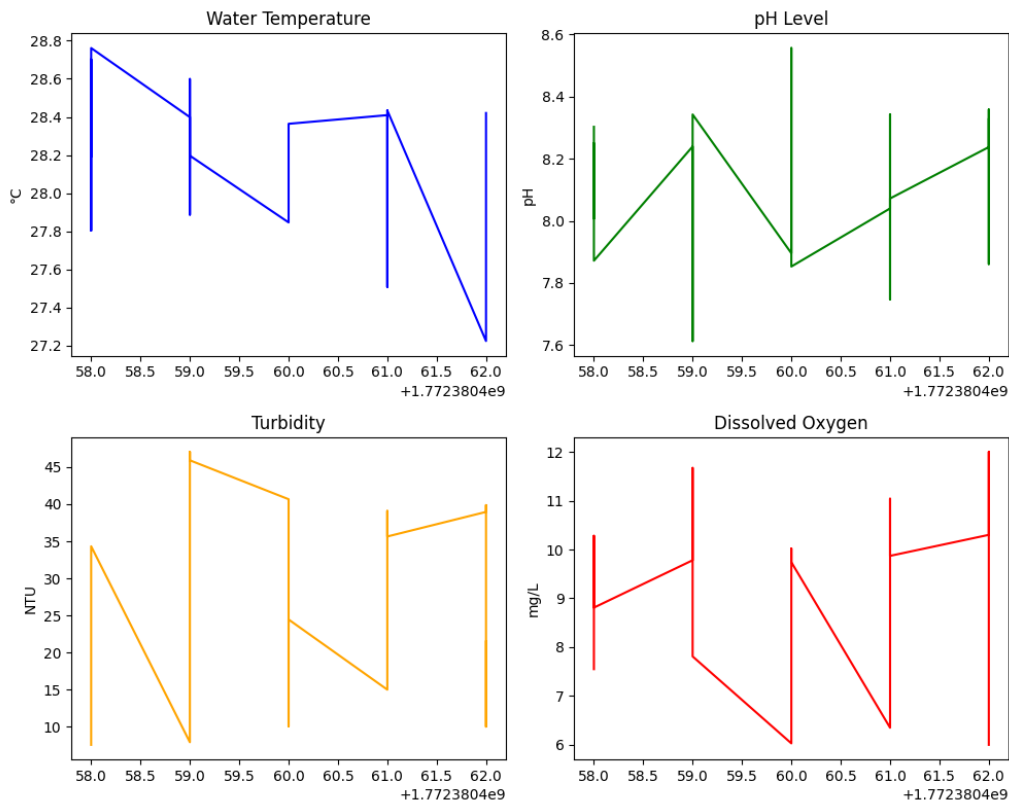


Figura 18: Grafica generada del sensor_id 6

Cada gráfico muestra las cuatro variables de calidad del agua para un sensor concreto a lo largo del tiempo. Las series presentan la variabilidad esperada en un sistema de monitorización real, con oscilaciones continuas en temperatura, pH, turbidez y oxígeno disuelto.

Parte 3: Integrando procesos ML en Data Streaming (2 puntos)

Ejercicio 3.1

Descarga el archivo `aeration_classifier.joblib`. Modifica el archivo `water_data_consumer_pr_2.ipynb` para realizar inferencia con el modelo de clasificación, que es un *RandomForest* que acepta como entrada `water_temperature`, `ph_level`, `turbidity` y `dissolved_oxygen_level`, y da un resultado (0,1) indicando si hay un proceso de aireación presente o no. Añade un productor que publique los resultados de inferencia, publicando la probabilidad de que el ejemplo pertenezca a cada una de las dos clases (mediante el uso de `predict_proba`). Imprime los resultados de inferencia por pantalla y adjunta una captura de la ejecución con 2 procesadores ML y múltiples productores activos (1 punto).

- Productor:

```

producers[i].close()

[producer=0] key=0 value={'sensor_id': '0', 'timestamp': 1772365654, 'water_temperature': 27.067830227886642, 'ph_level': 8.249775426395768, 'turbidity': 8.76, 'dissolved_oxygen': 6.17}
[producer=1] key=1 value={'sensor_id': '1', 'timestamp': 1772365654, 'water_temperature': 27.07193862420869, 'ph_level': 7.73102900983372, 'turbidity': 26.47, 'dissolved_oxygen': 5.21}
[producer=2] key=2 value={'sensor_id': '2', 'timestamp': 1772365654, 'water_temperature': 26.18923068345076, 'ph_level': 7.956339227462955, 'turbidity': 13.0, 'dissolved_oxygen': 11.18}
[producer=3] key=3 value={'sensor_id': '3', 'timestamp': 1772365654, 'water_temperature': 27.53892099509144, 'ph_level': 7.513300834897358, 'turbidity': 45.08, 'dissolved_oxygen': 8.46}
[producer=4] key=4 value={'sensor_id': '4', 'timestamp': 1772365654, 'water_temperature': 27.579917978935242, 'ph_level': 8.07983229041224, 'turbidity': 49.78, 'dissolved_oxygen': 9.93}
[producer=5] key=5 value={'sensor_id': '5', 'timestamp': 1772365654, 'water_temperature': 27.474725827878157, 'ph_level': 8.423084107502252, 'turbidity': 28.68, 'dissolved_oxygen': 11.1}
[producer=6] key=6 value={'sensor_id': '6', 'timestamp': 1772365654, 'water_temperature': 27.953736253030197, 'ph_level': 7.960693265358334, 'turbidity': 24.6, 'dissolved_oxygen': 8.43}
[producer=0] key=0 value={'sensor_id': '0', 'timestamp': 1772365655, 'water_temperature': 26.806683032914908, 'ph_level': 8.26914175692359, 'turbidity': 10.38, 'dissolved_oxygen': 11.12}
[producer=1] key=1 value={'sensor_id': '1', 'timestamp': 1772365655, 'water_temperature': 26.112236944584584, 'ph_level': 7.717543315051939, 'turbidity': 18.92, 'dissolved_oxygen': 11.73}
[producer=2] key=2 value={'sensor_id': '2', 'timestamp': 1772365655, 'water_temperature': 26.4812173792703, 'ph_level': 7.795015178029417, 'turbidity': 46.27, 'dissolved_oxygen': 8.28}
[producer=3] key=3 value={'sensor_id': '3', 'timestamp': 1772365655, 'water_temperature': 26.79798124815939, 'ph_level': 7.677320653679724, 'turbidity': 20.58, 'dissolved_oxygen': 10.94}
[producer=4] key=4 value={'sensor_id': '4', 'timestamp': 1772365655, 'water_temperature': 26.04733090544926, 'ph_level': 8.15456403462893, 'turbidity': 39.73, 'dissolved_oxygen': 9.03}
[producer=5] key=5 value={'sensor_id': '5', 'timestamp': 1772365655, 'water_temperature': 27.949418305939393, 'ph_level': 7.54680450319697, 'turbidity': 38.69, 'dissolved_oxygen': 11.21}
[producer=6] key=6 value={'sensor_id': '6', 'timestamp': 1772365655, 'water_temperature': 27.522067845710623, 'ph_level': 8.3900794905824, 'turbidity': 15.52, 'dissolved_oxygen': 11.0}
[producer=0] key=0 value={'sensor_id': '0', 'timestamp': 1772365656, 'water_temperature': 26.87916949616388, 'ph_level': 8.329410780270639, 'turbidity': 30.12, 'dissolved_oxygen': 6.07}
[producer=1] key=1 value={'sensor_id': '1', 'timestamp': 1772365656, 'water_temperature': 27.945581650751173, 'ph_level': 8.451198307716602, 'turbidity': 33.73, 'dissolved_oxygen': 11.45}

```

Figura 19: Salida del productor.

■ Consumidor 1 y 2:

```

producer.close()
Received: {'sensor_id': '0', 'timestamp': 1772365654, 'water_temperature': 27.067830227886642, 'p_h_level': 8.249775426395768, 'turbidity': 8.76, 'dissolved_oxygen': 6.17} Key: 0
[ML -> ML_2] key=0 pred=1 proba=[0.0, 1.0] timestamp=1772365654
Received: {'sensor_id': '2', 'timestamp': 1772365654, 'water_temperature': 26.18923068345076, 'ph_level': 7.956339227462955, 'turbidity': 13.0, 'dissolved_oxygen': 11.18} Key: 2
[ML -> ML_2] key=2 pred=0 proba=[1.0, 0.0] timestamp=1772365654
Received: {'sensor_id': '5', 'timestamp': 1772365654, 'water_temperature': 27.474725827878157, 'p_h_level': 8.423084107502252, 'turbidity': 28.68, 'dissolved_oxygen': 11.1} Key: 5
[ML -> ML_2] key=5 pred=0 proba=[1.0, 0.0] timestamp=1772365654
Received: {'sensor_id': '6', 'timestamp': 1772365654, 'water_temperature': 27.953736253030197, 'p_h_level': 7.960693265358334, 'turbidity': 24.6, 'dissolved_oxygen': 8.43} Key: 6
[ML -> ML_2] key=6 pred=0 proba=[0.8, 0.2] timestamp=1772365654
Received: {'sensor_id': '0', 'timestamp': 1772365655, 'water_temperature': 26.806683032914908, 'p_h_level': 8.26914175692359, 'turbidity': 10.38, 'dissolved_oxygen': 11.12} Key: 0
[ML -> ML_2] key=0 pred=0 proba=[1.0, 0.0] timestamp=1772365655
Received: {'sensor_id': '2', 'timestamp': 1772365655, 'water_temperature': 26.4812173792703, 'ph_level': 7.795015178029417, 'turbidity': 46.27, 'dissolved_oxygen': 8.28} Key: 2
[ML -> ML_2] key=2 pred=0 proba=[0.7, 0.3] timestamp=1772365655
Received: {'sensor_id': '5', 'timestamp': 1772365655, 'water_temperature': 27.949418305939393, 'p_h_level': 7.54680450319697, 'turbidity': 38.69, 'dissolved_oxygen': 11.21} Key: 5
[ML -> ML_2] key=5 pred=0 proba=[1.0, 0.0] timestamp=1772365655
Received: {'sensor_id': '6', 'timestamp': 1772365655, 'water_temperature': 27.522067845710623, 'p_h_level': 8.3900794905824, 'turbidity': 15.52, 'dissolved_oxygen': 11.0} Key: 6
[ML -> ML_2] key=6 pred=0 proba=[1.0, 0.0] timestamp=1772365655
Received: {'sensor_id': '0', 'timestamp': 1772365656, 'water_temperature': 26.87916949616388, 'ph_level': 8.329410780270639, 'turbidity': 30.12, 'dissolved_oxygen': 6.07} Key: 0
[ML -> ML_2] key=0 pred=1 proba=[0.0, 1.0] timestamp=1772365656
Received: {'sensor_id': '2', 'timestamp': 1772365656, 'water_temperature': 27.241979906828047, 'p_h_level': 7.876709799512081, 'turbidity': 11.2, 'dissolved_oxygen': 9.17} Key: 2
[ML -> ML_2] key=2 pred=0 proba=[1.0, 0.0] timestamp=1772365656
Received: {'sensor_id': '6', 'timestamp': 1772365656, 'water_temperature': 26.007658652182325, 'p_h_level': 8.424400806816156, 'turbidity': 17.1, 'dissolved_oxygen': 9.95} Key: 6
[ML -> ML_2] key=6 pred=0 proba=[1.0, 0.0] timestamp=1772365656
Received: {'sensor_id': '5', 'timestamp': 1772365656, 'water_temperature': 26.36350041360607, 'ph_level': 7.684752014936809, 'turbidity': 13.71, 'dissolved_oxygen': 5.59} Key: 5
[ML -> ML_2] key=5 pred=1 proba=[0.0, 1.0] timestamp=1772365656
Received: {'sensor_id': '0', 'timestamp': 1772365657, 'water_temperature': 27.05916452881824, 'ph_level': 8.44614653746677, 'turbidity': 37.26, 'dissolved_oxygen': 5.22} Key: 0

producer.close()
Received: {'sensor_id': '1', 'timestamp': 1772365654, 'water_temperature': 27.07193862420869, 'ph_level': 7.73102900983372, 'turbidity': 26.47, 'dissolved_oxygen': 5.21} Key: 1
[ML -> ML_1] key=1 pred=1 proba=[0.0, 1.0] timestamp=1772365654
Received: {'sensor_id': '3', 'timestamp': 1772365654, 'water_temperature': 27.53892099509144, 'ph_level': 7.513300834897358, 'turbidity': 45.08, 'dissolved_oxygen': 8.46} Key: 3
[ML -> ML_1] key=3 pred=0 proba=[0.5, 0.5] timestamp=1772365654
Received: {'sensor_id': '4', 'timestamp': 1772365654, 'water_temperature': 27.579917978935242, 'p_h_level': 8.07983229041224, 'turbidity': 49.78, 'dissolved_oxygen': 9.93} Key: 4
[ML -> ML_1] key=4 pred=0 proba=[1.0, 0.0] timestamp=1772365654
Received: {'sensor_id': '1', 'timestamp': 1772365655, 'water_temperature': 26.112236944584584, 'p_h_level': 7.717543315051939, 'turbidity': 18.92, 'dissolved_oxygen': 11.73} Key: 1
[ML -> ML_1] key=1 pred=0 proba=[1.0, 0.0] timestamp=1772365655
Received: {'sensor_id': '3', 'timestamp': 1772365655, 'water_temperature': 26.79798124815939, 'ph_level': 7.677320653679724, 'turbidity': 20.58, 'dissolved_oxygen': 10.94} Key: 3
[ML -> ML_1] key=3 pred=0 proba=[1.0, 0.0] timestamp=1772365655
Received: {'sensor_id': '4', 'timestamp': 1772365655, 'water_temperature': 26.04733090544926, 'ph_level': 8.15456403462893, 'turbidity': 39.73, 'dissolved_oxygen': 9.03} Key: 4
[ML -> ML_1] key=4 pred=0 proba=[1.0, 0.0] timestamp=1772365655
Received: {'sensor_id': '5', 'timestamp': 1772365655, 'water_temperature': 27.949418305939393, 'p_h_level': 7.54680450319697, 'turbidity': 38.69, 'dissolved_oxygen': 11.21} Key: 5
[ML -> ML_1] key=5 pred=0 proba=[1.0, 0.0] timestamp=1772365655
Received: {'sensor_id': '6', 'timestamp': 1772365655, 'water_temperature': 27.522067845710623, 'p_h_level': 8.3688212873544, 'turbidity': 37.85, 'dissolved_oxygen': 10.53} Key: 6
[ML -> ML_1] key=6 pred=0 proba=[1.0, 0.0] timestamp=1772365655
Received: {'sensor_id': '4', 'timestamp': 1772365656, 'water_temperature': 26.886228811260555, 'p_h_level': 7.541365408798342, 'turbidity': 47.58, 'dissolved_oxygen': 8.31} Key: 4
[ML -> ML_1] key=4 pred=1 proba=[0.4, 0.6] timestamp=1772365656
Received: {'sensor_id': '1', 'timestamp': 1772365656, 'water_temperature': 27.945581650751173, 'p_h_level': 8.451198307716602, 'turbidity': 33.73, 'dissolved_oxygen': 11.45} Key: 1
[ML -> ML_1] key=1 pred=0 proba=[1.0, 0.0] timestamp=1772365656
Received: {'sensor_id': '3', 'timestamp': 1772365657, 'water_temperature': 27.73630886768602, 'ph_level': 8.453374748436202, 'turbidity': 25.37, 'dissolved_oxygen': 9.09} Key: 3
[ML -> ML_1] key=3 pred=0 proba=[1.0, 0.0] timestamp=1772365657
Received: {'sensor_id': '1', 'timestamp': 1772365657, 'water_temperature': 27.720828821882048, 'p_h_level': 7.54316215855283, 'turbidity': 48.33, 'dissolved_oxygen': 7.07} Key: 1
[ML -> ML_1] key=1 pred=1 proba=[0.0, 1.0] timestamp=1772365657
Received: {'sensor_id': '4', 'timestamp': 1772365657, 'water_temperature': 27.831004723630787, 'p_h_level': 8.20003707761275, 'turbidity': 17.65, 'dissolved_oxygen': 9.69} Key: 4
[ML -> ML_1] key=4 pred=0 proba=[1.0, 0.0] timestamp=1772365657
Received: {'sensor_id': '1', 'timestamp': 1772365658, 'water_temperature': 26.19411717613735, 'ph_level': 8.399149750677614, 'turbidity': 31.18, 'dissolved_oxygen': 10.6} Key: 1

```

Figura 20: Salida de los consumidores ML.

En la salida del consumidor ML se puede observar, para cada mensaje procesado, el `sensor_id`, el `timestamp`, las probabilidades `p_class_0` y `p_class_1`, y la predicción final (`pred`). Los resultados son coherentes: cuando `p_class_1` es dominante, se predice aireación activa, y viceversa.

Ejercicio 3.3

Crea otro notebook con un consumidor que se suscriba a los resultados de inferencia publicados por el productor del apartado 3.2 y que los agregue en tres gráficos para cada sensor:

3.3.1 Estado de aireación para cada timestamp.

3.3.2 Confianza de la clase de aireación por timestamp cuando es la clase escogida.

3.3.3 Confianza de la clase de *no* aireación por timestamp cuando es la clase escogida.

Imprime por pantalla los resultados de inferencia y adjunta una captura de la ejecución con 2 procesadores ML y múltiples productores activos. Adjunta imágenes de todos los gráficos generados (1 punto).

Se ha creado el notebook `ml_data_consumer_pr_2.ipynb`, que actúa como consumidor del tópico de resultados de inferencia. Para cada `sensor_id`, mantiene tres series temporales independientes y genera tres subplots:

- **Estado de aireación** (azul): valor binario 0 o 1 predicho por el modelo en cada instante. Permite ver la frecuencia y duración de los eventos de aireación.
- **Confianza clase 0/no aireación** (verde): probabilidad asignada a la clase 0 en los instantes en que es la clase predicha. Refleja la certeza del modelo cuando decide que no hay aireación.
- **Confianza clase 1/aireación** (amarillo): probabilidad asignada a la clase 1 en los instantes en que es la clase predicha. Permite valorar cuán seguro está el modelo cuando detecta aireación.

Esta separación entre estado y confianza es muy relevante en aplicaciones industriales: una predicción de aireación con baja confianza puede requerir supervisión manual, mientras que una alta confianza en ambas clases permite automatizar completamente la respuesta del sistema.

- **Productor (sensores del 0 al 6):**

```
Producing messages to Kafka topic 'water_quality'...
Sent0, {'timestamp': 1772387419, 'water_temperature': 30.087966589910195, 'ph_level': 8.3194462765083, 'turbidity': 39.55, 'dissolved_oxygen': 8.41}
Sent1, {'timestamp': 1772387419, 'water_temperature': 28.98037068155586, 'ph_level': 8.258523427505168, 'turbidity': 34.41, 'dissolved_oxygen': 6.02}
Sent2, {'timestamp': 1772387419, 'water_temperature': 28.421226361332003, 'ph_level': 7.98611729167859, 'turbidity': 37.74, 'dissolved_oxygen': 6.45}
Sent3, {'timestamp': 1772387419, 'water_temperature': 29.20893981897406, 'ph_level': 8.637922952410127, 'turbidity': 42.08, 'dissolved_oxygen': 8.92}
Sent4, {'timestamp': 1772387419, 'water_temperature': 28.66995604001854, 'ph_level': 8.5820170268147, 'turbidity': 14.84, 'dissolved_oxygen': 8.32}
Sent5, {'timestamp': 1772387419, 'water_temperature': 28.969429586872455, 'ph_level': 7.755849828291856, 'turbidity': 35.73, 'dissolved_oxygen': 5.49}
Sent6, {'timestamp': 1772387419, 'water_temperature': 29.2760398054686, 'ph_level': 8.354251820693085, 'turbidity': 22.02, 'dissolved_oxygen': 5.59}
Sent0, {'timestamp': 1772387420, 'water_temperature': 28.72128001662278, 'ph_level': 7.760107418566658, 'turbidity': 11.43, 'dissolved_oxygen': 5.44}
Sent1, {'timestamp': 1772387420, 'water_temperature': 28.90220170249759, 'ph_level': 7.9772058177951095, 'turbidity': 14.0, 'dissolved_oxygen': 8.34}
Sent2, {'timestamp': 1772387420, 'water_temperature': 28.678574381954533, 'ph_level': 8.621795390642399, 'turbidity': 37.39, 'dissolved_oxygen': 7.54}
Sent3, {'timestamp': 1772387420, 'water_temperature': 29.330732753463426, 'ph_level': 8.532395763619279, 'turbidity': 42.93, 'dissolved_oxygen': 9.46}
Sent4, {'timestamp': 1772387420, 'water_temperature': 28.75259052099138, 'ph_level': 7.894906752357813, 'turbidity': 17.29, 'dissolved_oxygen': 6.25}
Sent5, {'timestamp': 1772387420, 'water_temperature': 30.264167698188935, 'ph_level': 7.889467686118424, 'turbidity': 26.12, 'dissolved_oxygen': 9.62}
Sent6, {'timestamp': 1772387420, 'water_temperature': 28.805864602218325, 'ph_level': 8.424178594503985, 'turbidity': 33.39, 'dissolved_oxygen': 8.04}
Sent0, {'timestamp': 1772387421, 'water_temperature': 30.18512220348991, 'ph_level': 8.595901302713937, 'turbidity': 37.16, 'dissolved_oxygen': 11.25}
Sent1, {'timestamp': 1772387421, 'water_temperature': 30.322332360110877, 'ph_level': 7.7622171597804135, 'turbidity': 6.51, 'dissolved_oxygen': 11.73}
Sent2, {'timestamp': 1772387421, 'water_temperature': 30.024284641607004, 'ph_level': 8.566099001123362, 'turbidity': 22.44, 'dissolved_oxygen': 11.49}
Sent3, {'timestamp': 1772387421, 'water_temperature': 28.541662003751057, 'ph_level': 8.541035102613066, 'turbidity': 6.68, 'dissolved_oxygen': 7.62}
Sent4, {'timestamp': 1772387421, 'water_temperature': 28.902498076801706, 'ph_level': 8.088865994307795, 'turbidity': 46.0, 'dissolved_oxygen': 7.4}
Sent5, {'timestamp': 1772387421, 'water_temperature': 28.63380681630891, 'ph_level': 7.980944921135632, 'turbidity': 32.6, 'dissolved_oxygen': 7.72}
Sent6, {'timestamp': 1772387421, 'water_temperature': 29.610014853026428, 'ph_level': 7.701659624909834, 'turbidity': 16.11, 'dissolved_oxygen': 7.57}
```

Figura 21: Salida del productor.

- **Consumidor 1 (ML A):**

```

consumer.close()
producer.close()

Received: {'sensor_id': '1', 'timestamp': 1772370738, 'water_temperature': 26.749950447938797, 'ph_level': 8.269890917106068, 'turbidity': 8.1, 'dissolved_oxygen': 8.99} Key: 1
[ML -> ML_2] key=1 pred=0 proba=[0.9, 0.1] timestamp=1772370738
Received: {'sensor_id': '4', 'timestamp': 1772370738, 'water_temperature': 26.79211012678646, 'ph_level': 8.48143159130922, 'turbidity': 33.95, 'dissolved_oxygen': 6.71} Key: 4
[ML -> ML_2] key=4 pred=1 proba=[0.0, 1.0] timestamp=1772370738
Received: {'sensor_id': '3', 'timestamp': 1772370738, 'water_temperature': 26.71690577749418, 'ph_level': 7.740399371907019, 'turbidity': 13.23, 'dissolved_oxygen': 11.94} Key: 3
[ML -> ML_2] key=3 pred=0 proba=[1.0, 0.0] timestamp=1772370738
Received: {'sensor_id': '1', 'timestamp': 1772370739, 'water_temperature': 27.72372454024529, 'ph_level': 7.7309444785542778, 'turbidity': 11.1, 'dissolved_oxygen': 8.54} Key: 1
[ML -> ML_2] key=1 pred=0 proba=[0.7, 0.3] timestamp=1772370739
Received: {'sensor_id': '3', 'timestamp': 1772370739, 'water_temperature': 27.39420187279031, 'ph_level': 7.906304441867583, 'turbidity': 37.19, 'dissolved_oxygen': 7.83} Key: 3
[ML -> ML_2] key=3 pred=0 proba=[0.5, 0.5] timestamp=1772370739
Received: {'sensor_id': '4', 'timestamp': 1772370739, 'water_temperature': 26.564652869028432, 'ph_level': 8.366346441326286, 'turbidity': 30.56, 'dissolved_oxygen': 5.88} Key: 4
[ML -> ML_2] key=4 pred=1 proba=[0.0, 1.0] timestamp=1772370739
Received: {'sensor_id': '1', 'timestamp': 1772370740, 'water_temperature': 27.61111427673508, 'ph_level': 7.62873602276108, 'turbidity': 29.34, 'dissolved_oxygen': 10.59} Key: 1
[ML -> ML_2] key=1 pred=0 proba=[1.0, 0.0] timestamp=1772370740
Received: {'sensor_id': '3', 'timestamp': 1772370740, 'water_temperature': 26.691257005010264, 'ph_level': 8.471911500227641, 'turbidity': 7.64, 'dissolved_oxygen': 9.99} Key: 3
[ML -> ML_2] key=3 pred=0 proba=[0.8, 0.2] timestamp=1772370740
Received: {'sensor_id': '4', 'timestamp': 1772370740, 'water_temperature': 26.55194989140394, 'ph_level': 8.323024251049063, 'turbidity': 49.17, 'dissolved_oxygen': 10.88} Key: 4
[ML -> ML_2] key=4 pred=0 proba=[1.0, 0.0] timestamp=1772370740
Received: {'sensor_id': '0', 'timestamp': 1772370733, 'water_temperature': 27.37902879470378, 'ph_level': 8.178525934826405, 'turbidity': 8.19, 'dissolved_oxygen': 6.44} Key: 0
[ML -> ML_2] key=0 pred=1 proba=[0.0, 1.0] timestamp=1772370733
Received: {'sensor_id': '6', 'timestamp': 1772370733, 'water_temperature': 27.83557848944306, 'ph_level': 7.961121932218683, 'turbidity': 26.44, 'dissolved_oxygen': 5.95} Key: 6

```

Figura 22: Salida del consumidor 1 – ML A (ejercicio 3.3).

■ Consumidor 2 (ML B):

```

producer.close()

Received: {'sensor_id': '2', 'timestamp': 1772370791, 'water_temperature': 27.571726622946944, 'ph_level': 8.214336091072084, 'turbidity': 24.6, 'dissolved_oxygen': 10.04} Key: 2
[ML -> ML_1] key=2 pred=0 proba=[1.0, 0.0] timestamp=1772370791
Received: {'sensor_id': '0', 'timestamp': 1772370791, 'water_temperature': 26.81722917268189, 'ph_level': 8.041756120200109, 'turbidity': 42.01, 'dissolved_oxygen': 7.95} Key: 0
[ML -> ML_1] key=0 pred=0 proba=[0.5, 0.5] timestamp=1772370791
Received: {'sensor_id': '5', 'timestamp': 1772370791, 'water_temperature': 26.936239679987516, 'ph_level': 8.267044749103315, 'turbidity': 24.25, 'dissolved_oxygen': 7.7} Key: 5
[ML -> ML_1] key=5 pred=1 proba=[0.0, 1.0] timestamp=1772370791
Received: {'sensor_id': '6', 'timestamp': 1772370791, 'water_temperature': 26.70316466267144, 'ph_level': 8.195655021140551, 'turbidity': 19.7, 'dissolved_oxygen': 5.61} Key: 6
[ML -> ML_1] key=6 pred=1 proba=[0.0, 1.0] timestamp=1772370791
Received: {'sensor_id': '2', 'timestamp': 1772370792, 'water_temperature': 27.716620123219677, 'ph_level': 7.527123021696313, 'turbidity': 46.95, 'dissolved_oxygen': 6.4} Key: 2
[ML -> ML_1] key=2 pred=1 proba=[0.1, 0.9] timestamp=1772370792
Received: {'sensor_id': '5', 'timestamp': 1772370792, 'water_temperature': 26.44991525725983, 'ph_level': 8.473324601573502, 'turbidity': 29.03, 'dissolved_oxygen': 5.41} Key: 5
[ML -> ML_1] key=5 pred=1 proba=[0.0, 1.0] timestamp=1772370792
Received: {'sensor_id': '0', 'timestamp': 1772370792, 'water_temperature': 26.26740455150006, 'ph_level': 8.217789653913389, 'turbidity': 18.34, 'dissolved_oxygen': 8.36} Key: 0
[ML -> ML_1] key=0 pred=0 proba=[0.6, 0.4] timestamp=1772370792
Received: {'sensor_id': '6', 'timestamp': 1772370792, 'water_temperature': 26.726189605921977, 'ph_level': 7.733023117383915, 'turbidity': 44.74, 'dissolved_oxygen': 11.71} Key: 6
[ML -> ML_1] key=6 pred=0 proba=[1.0, 0.0] timestamp=1772370792
Received: {'sensor_id': '0', 'timestamp': 1772370793, 'water_temperature': 27.49393199957361, 'ph_level': 7.849568825896254, 'turbidity': 14.56, 'dissolved_oxygen': 7.96} Key: 0
[ML -> ML_1] key=0 pred=0 proba=[0.6, 0.4] timestamp=1772370793
Received: {'sensor_id': '2', 'timestamp': 1772370793, 'water_temperature': 27.931081399754135, 'ph_level': 7.548118174011901, 'turbidity': 47.51, 'dissolved_oxygen': 9.18} Key: 2
[ML -> ML_1] key=2 pred=0 proba=[1.0, 0.0] timestamp=1772370793
Received: {'sensor_id': '6', 'timestamp': 1772370793, 'water_temperature': 27.25595194920987, 'ph_level': 8.133490458375185, 'turbidity': 22.98, 'dissolved_oxygen': 7.71} Key: 6
[ML -> ML_1] key=6 pred=1 proba=[0.2, 0.8] timestamp=1772370793
Received: {'sensor_id': '5', 'timestamp': 1772370793, 'water_temperature': 26.540197123311074, 'ph_level': 8.362455444883716, 'turbidity': 42.1, 'dissolved_oxygen': 7.71} Key: 5

```

Figura 23: Salida del consumidor 2 – ML B (ejercicio 3.3).

■ Nuevo Consumidor:

```

Subscribed to Kafka topic 'water_quality'.
Received: sensor_id:0,{ 'timestamp': 1772387570, 'water_temperature': 28.4620222259529, 'ph_level': 8.470267714857874, 'turbidity': 49.28, 'dissolved_oxygen': 7.63}
Datos del proceso de aireación ML B del sensor 0: { 'sensor_id': '0', 'timestamp': 1772387570, 'p_class_0': 0.2, 'p_class_1': 0.8, 'pred': 1}
Received: sensor_id:2,{ 'timestamp': 1772387570, 'water_temperature': 29.28788779416022, 'ph_level': 8.10778461092128, 'turbidity': 48.6, 'dissolved_oxygen': 5.42}
Datos del proceso de aireación ML B del sensor 2: { 'sensor_id': '2', 'timestamp': 1772387570, 'p_class_0': 0.0, 'p_class_1': 1.0, 'pred': 1}
Received: sensor_id:6,{ 'timestamp': 1772387570, 'water_temperature': 29.550104589299824, 'ph_level': 8.585891626509017, 'turbidity': 36.2, 'dissolved_oxygen': 7.53}
Datos del proceso de aireación ML B del sensor 6: { 'sensor_id': '6', 'timestamp': 1772387570, 'p_class_0': 0.7, 'p_class_1': 0.3, 'pred': 0}
Received: sensor_id:5,{ 'timestamp': 1772387570, 'water_temperature': 29.998125750347292, 'ph_level': 7.784929297680902, 'turbidity': 11.53, 'dissolved_oxygen': 7.06}
Datos del proceso de aireación ML B del sensor 5: { 'sensor_id': '5', 'timestamp': 1772387570, 'p_class_0': 0.1, 'p_class_1': 0.9, 'pred': 1}
Received: sensor_id:1,{ 'timestamp': 1772387570, 'water_temperature': 29.377275349237664, 'ph_level': 8.23932988911184, 'turbidity': 8.15, 'dissolved_oxygen': 10.19}
Datos del proceso de aireación ML B del sensor 1: { 'sensor_id': '1', 'timestamp': 1772387570, 'p_class_0': 1.0, 'p_class_1': 0.0, 'pred': 0}
Received: sensor_id:3,{ 'timestamp': 1772387570, 'water_temperature': 28.709796781549812, 'ph_level': 7.917089165243505, 'turbidity': 20.13, 'dissolved_oxygen': 6.16}
Datos del proceso de aireación ML B del sensor 3: { 'sensor_id': '3', 'timestamp': 1772387570, 'p_class_0': 0.0, 'p_class_1': 1.0, 'pred': 1}
Received: sensor_id:4,{ 'timestamp': 1772387570, 'water_temperature': 29.48201088190075, 'ph_level': 8.104505795636673, 'turbidity': 14.83, 'dissolved_oxygen': 6.38}
Datos del proceso de aireación ML B del sensor 4: { 'sensor_id': '4', 'timestamp': 1772387570, 'p_class_0': 0.0, 'p_class_1': 1.0, 'pred': 1}
Received: sensor_id:0,{ 'timestamp': 1772387571, 'water_temperature': 29.0333444405339, 'ph_level': 8.551834910446305, 'turbidity': 39.56, 'dissolved_oxygen': 7.93}
Datos del proceso de aireación ML B del sensor 0: { 'sensor_id': '0', 'timestamp': 1772387571, 'p_class_0': 0.5, 'p_class_1': 0.5, 'pred': 0}

```

Figura 24: Salida Nuevo Consumidor.

■ Gráficas de paneles de control

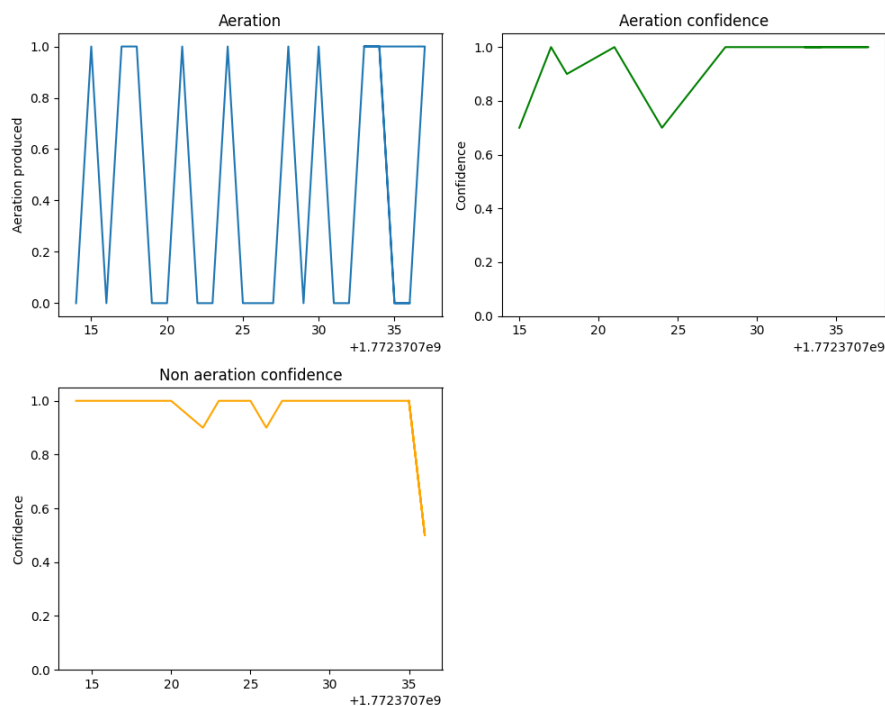


Figura 25: Panel de control – Sensor ID 0.

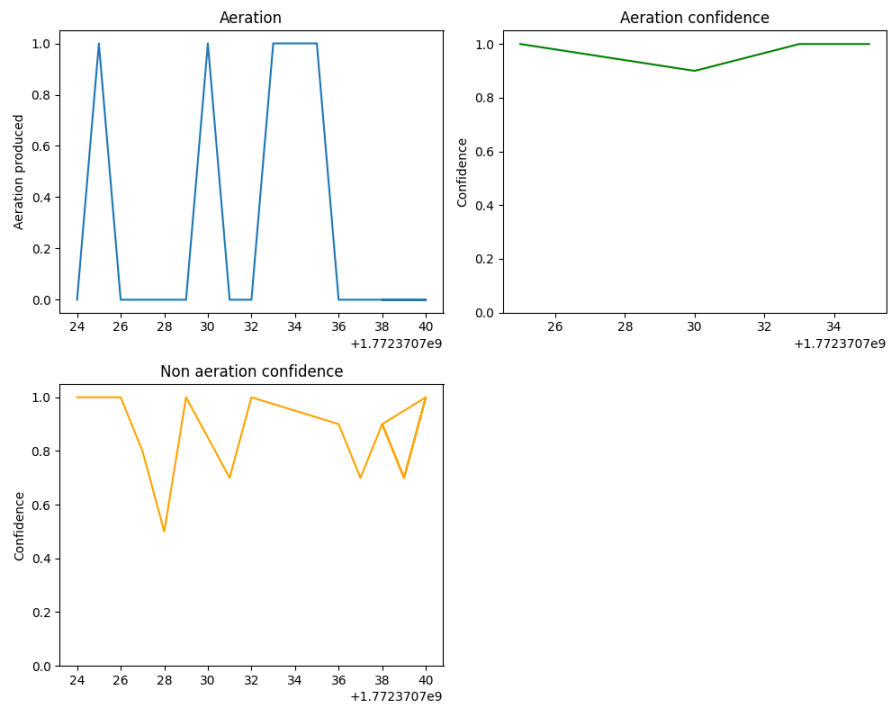


Figura 26: Panel de control – Sensor ID 1.

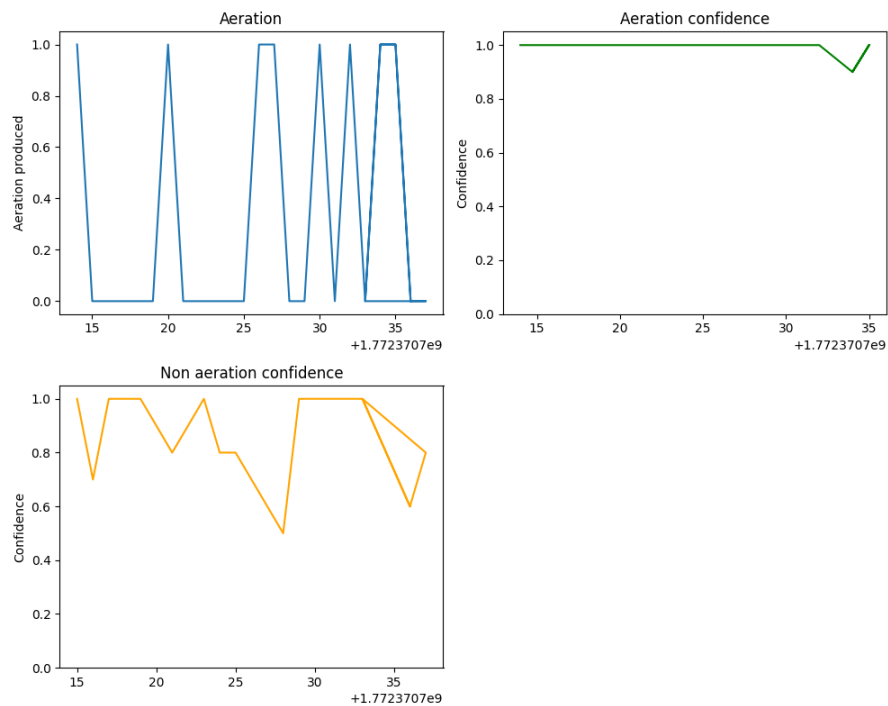


Figura 27: Panel de control – Sensor ID 2.

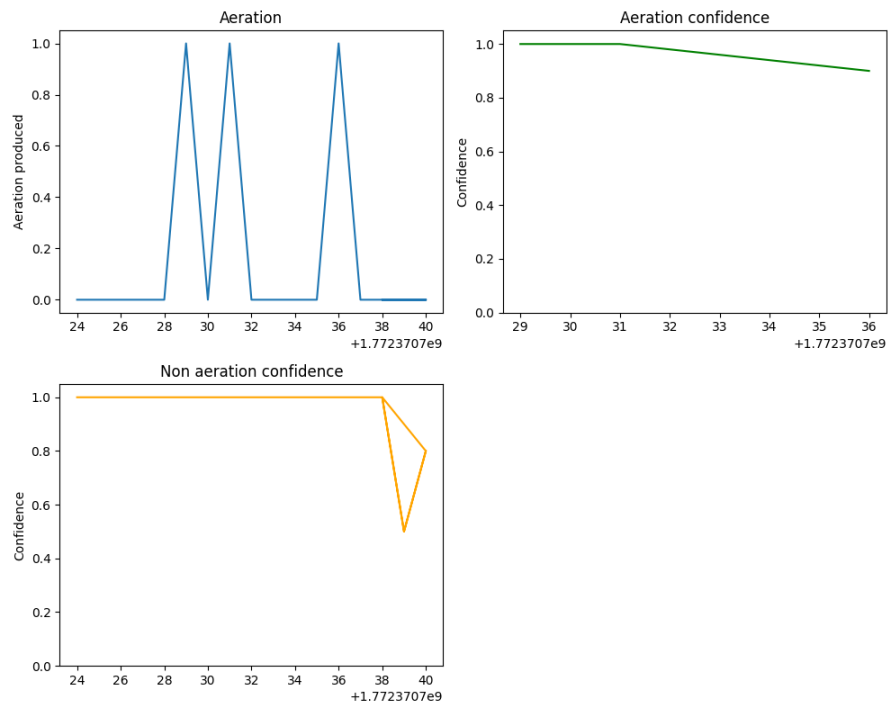


Figura 28: Panel de control – Sensor ID 3.

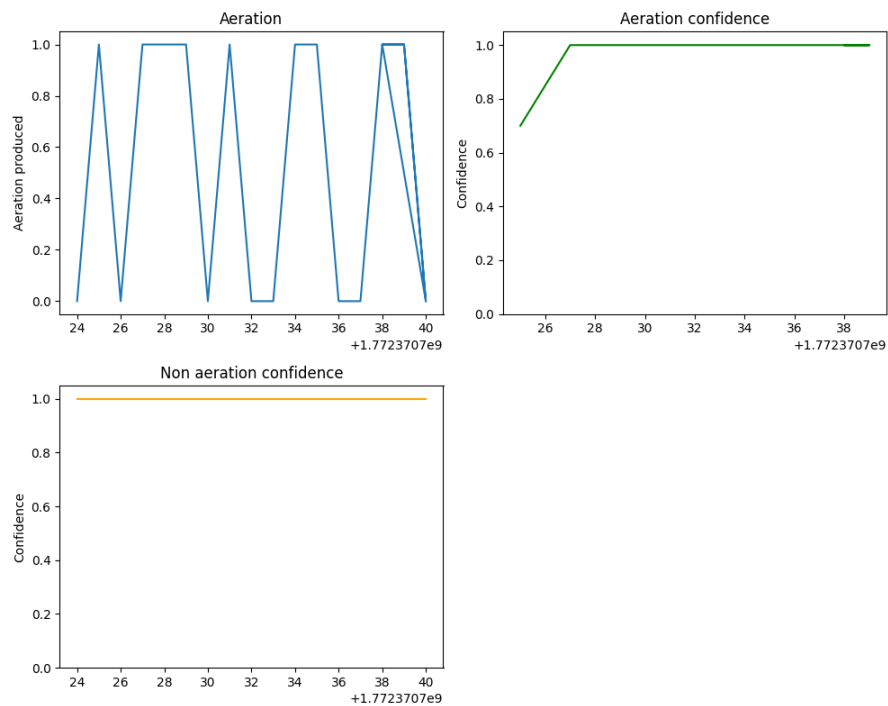


Figura 29: Panel de control – Sensor ID 4.

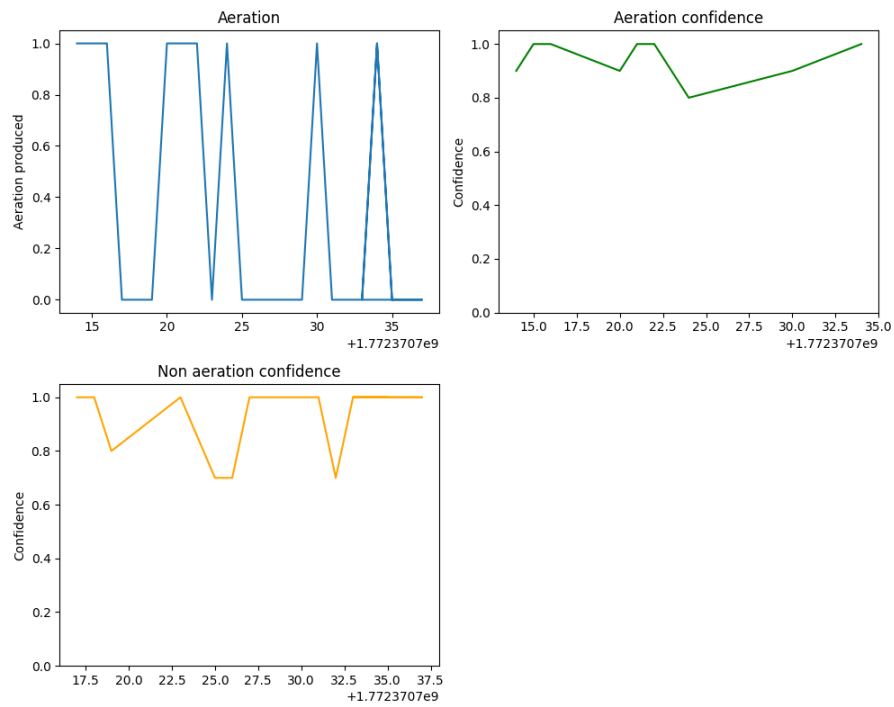


Figura 30: Panel de control – Sensor ID 5.

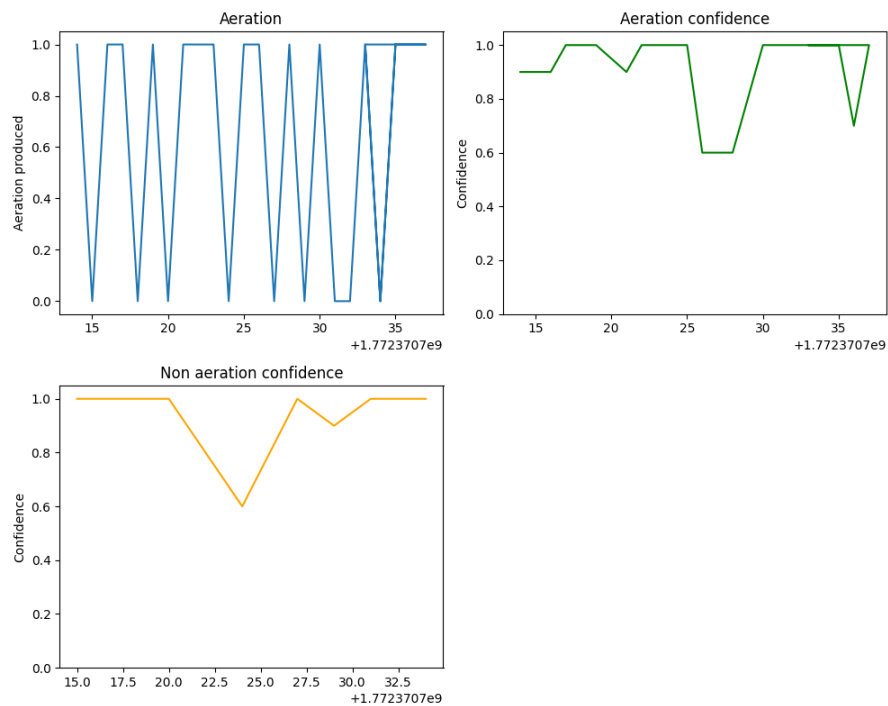


Figura 31: Panel de control – Sensor ID 6.